



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس شبکه‌های عصبی و یادگیری عمیق
پاسخ تمرین شماره یک

دانشجویان:

سوال یک و سه: احمدرضا نوپوش شماره دانشجویی ۶۱۰۳۰۱۱۹۴

سوال دو و چهار: رضا مردانی شماره دانشجویی: ۶۱۰۳۰۱۱۸۰

فهرست مطالب

۴	فهرست تصاویر
۶	فهرست جداول
۷	۱ نرون MacCulloch-Pitts
۷	۱-۱ منطق آستانه‌ای
۷	۱-۱-۱ گیت NOT
۸	۱-۱-۲ گیت AND
۹	۱-۱-۳ گیت OR
۱۰	۱-۱-۴ چرا یک نرون آستانه‌ای فقط می‌تواند توابع جداسازی‌پذیر خطی را نمایش دهد؟
۱۱	۱-۲ Majority و Parity
۱۱	۱-۲-۱ توجیه هندسی عدم امکان نمایش با یک نرون
۱۲	۱-۲-۲ طراحی شبکه برای تابع Parity
۱۳	۱-۲-۳ طراحی شبکه برای تابع Parity
۱۵	۲ Madaline و Adaline
۱۵	۲-۱
۱۵	۲-۱-۱ مشتق‌گیری و استخراج روابط، SSE LMS و منظم‌سازی Ridge
۱۹	۲-۱-۲ آمادسازی داده‌ها و تحلیل
۱۹	۲-۱-۳ ارزیابی و مصورسازی
۲۱	۲-۱-۴ مقایسه با مدل‌های پایه
۲۳	۲-۲ madaline
۲۳	۲-۲-۱ شبه‌کد
۲۴	۲-۲-۲ ارزیابی و نمودارها
۲۷	۲-۲-۳ مطالعه ابرپارامترها

۲۹	۳ رگرشن با استفاده از mlp
۲۹	۱-۳ پیش‌پردازش داده‌ها
۲۹	۱-۱-۳ مدیریت values missing
۳۰	۲-۱-۳ حذف کردن outliers از قسمت قیمت
۳۱	۳-۱-۳ تبدیل متغیرهای کیفی به یک مقدار قابل قبول
۳۲	۲-۳ پیاده‌سازی شبکه عصبی پیشنهادشده
۳۲	۳-۳ معیارهای سنجش مدل‌های رگرسیونی
۳۳	۱-۳-۳ میانگین مربعات خطا (Mean Squared Error - MSE)
۳۳	۲-۳-۳ ریشه میانگین مربعات خطا (Root Mean Squared Error - RMSE)
۳۴	۳-۳-۳ میانگین قدر مطلق خطا (Mean Absolute Error - MAE)
۳۴	۴-۳-۳ ضریب تعیین (R^2 Score)
۳۶	۴-۳ پیاده‌سازی شبکه عصبی پیچیده‌تر
۳۸	۵-۳ مقایسه و تحلیل نمودارها
۴۰	۴ طراحی شبکه AutoEncoder
۴۰	۱-۴ پیش‌پردازش
۴۰	۲-۴ پیاده‌سازی
۴۳	۳-۴ نتایج آموزش شبکه
۴۵	۴-۴ ارزیابی شبکه
۴۵	۱-۴-۴ خوشه‌بندی با الگوریتم K-Means
۴۵	۲-۴-۴ تحلیل خوشه‌ها
۴۷	۳-۴-۴ کاهش ابعاد با t-SNE
۴۸	۴-۴-۴ مقایسه با داده‌های خام
۵۰	۵-۴ معیارهای ارزیابی کیفیت خوشه‌بندی

فهرست تصاویر

۷	نورون آستانه‌ای NOT	۱
۸	نورون آستانه‌ای AND	۲
۹	نورون آستانه‌ای OR	۳
۱۲	شبکه نورون آستانه‌ای برای تابع Parity	۴
۱۴	نورون آستانه‌ای برای تابع Majority	۵
۲۱	pca	۶
۲۱	نمودار کاهش خطای Adaline	۷
۲۴	مرز تصمیم داده ماه	۸
۲۵	مرز تصمیم داده دایره	۹
۲۵	لایه‌های پنهان داده ماه	۱۰
۲۶	لایه‌های پنهان داده دایره	۱۱
۲۷	نمودار کاهش خطای داده ماه	۱۲
۲۷	نمودار کاهش خطای داده دایره	۱۳
۳۰	کورلیشن بین دو ستون	۱۴
۳۱	دیتاست قبل و بعد حذف outlier	۱۵
۳۶	نمودار کاهش خطای داده‌های آموزش، ارزیابی و آزمون روی شبکه عصبی ساده	۱۶
۳۷	نمودار کاهش خطای داده‌های آموزش، ارزیابی و آزمون روی شبکه عصبی پیچیده	۱۷
۳۸	مقایسه نمودار کاهش خطای بین دو مدل ساده و پیچیده	۱۸
۳۹	مقایسه مقدار واقعی با مقدار پیش‌بینی شده در دو مدل	۱۹
۴۳	نمودار کاهش خطای شبکه AutoEncoder	۲۰
۴۵	K-means	۲۱
۴۶	ماتریس در هم‌ریختگی بین خوشه‌ها و برچسب‌ها	۲۲
۴۷	t-SNE	۲۳
۴۸	kmeans داده‌های خام	۲۴

۴۹	 ماتریس درهمریختگی روی داده‌های خام	۲۵
۵۰	 t-SNE داده‌های خام	۲۶

فهرست جداول

۷	جدول درستی گیت NOT	۱
۸	جدول درستی گیت AND	۲
۹	جدول درستی گیت OR	۳
۱۳	خروجی شبکه برای ورودی‌های تابع Parity	۴
۱۹	آماره‌های داده‌ها قبل و بعد از نرمال‌سازی	۵
۱۹	توازن کلاس‌ها	۶
۲۰	ماتریس درهم‌ریختگی مدل Adaline روی مجموعه تست	۷
۲۰	معیارهای سنجش مدل Adaline روی داده آزمون	۸
۲۱	مقایسه‌ی عملکرد مدل‌ها بر اساس معیارهای مختلف	۹
۲۴	کارایی بهترین مدل روی داده‌های ماه و دایره	۱۰
۲۴	ماتریس درهم‌ریختگی برای داده ماه	۱۱
۲۴	ماتریس درهم‌ریختگی برای داده دایره	۱۲
۲۸	کارایی مدل‌ها بر اساس seed روی داده‌های ماه و دایره	۱۳
۳۲	معماری شبکه عصبی ساده	۱۴
۳۲	هابرپارامترهای مدل پیشنهادشده	۱۵
۳۵	مقایسه معیارهای ارزیابی مدل	۱۶
۳۵	معیارهای سنجش روی شبکه عصبی ساده	۱۷
۳۶	خلاصه معماری مدل پیچیده‌تر	۱۸
۳۷	هابرپارامترهای مدل پیچیده‌تر	۱۹
۳۷	معیارهای سنجش روی شبکه عصبی پیچیده‌تر	۲۰
۳۸	مقایسه نتایج ارزیابی مدل‌ها بر روی داده آزمون	۲۱
۴۲	معماری AutoEncoder	۲۲
۴۲	هابرپارامترهای مدل AutoEncoder	۲۳

۱ نرون MacCulloch-Pitts

در این بخش به پرسش‌های مربوط به نرون MacCulloch-Pitts پاسخ می‌دهیم. در بخش نخست، برای گیت‌های منطقی AND و OR و NOT یک نرون MacCulloch-Pitts طراحی می‌کنیم و تحلیل خواهیم کرد چرا این نرون آستانه‌ای تنها می‌تواند توابع بولی جداسازی‌پذیر خطی را جدا کند. در بخش دوم، یک شبکه دولایه‌ای برای توابع Majority و Parity طراحی خواهیم کرد و نشان خواهیم داد که چرا این طراحی درست است.

۱-۱ منطق آستانه‌ای

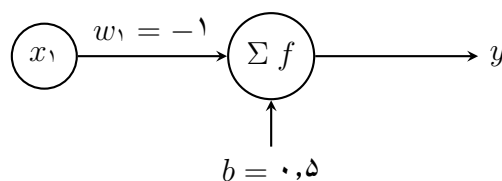
در این بخش ابتدا جدول درستی تابع بولی را ارائه می‌کنیم، سپس نرون آستانه‌ای طراحی شده را رسم کرده و نشان می‌دهیم که به ازای هر ورودی، نرون آستانه‌ای طراحی شده جواب مطلوب را که مطابق است با مقدار جواب جدول درستی - خروجی می‌دهد.

۱-۱-۱ گیت NOT

جدول ۱: جدول درستی گیت NOT

y	x_1
۱	۰
۰	۱

شکل ۱: نرون آستانه‌ای NOT



محاسبات به صورت زیر است:

• ورودی ۰:

$$z = (-1) \times 0 + 0.5 = 0.5 \geq 0 \rightarrow y = 1$$

• ورودی ۱:

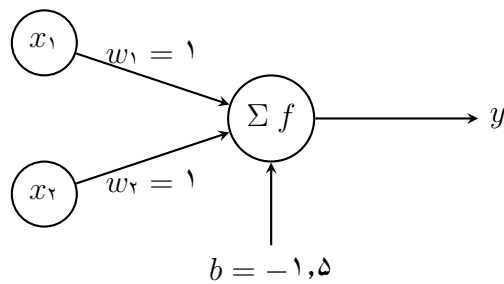
$$z = (-1) \times 1 + 0,5 = -0,5 < 0 \rightarrow y = 0$$

۲-۱-۱ گیت AND

جدول ۲: جدول درستی گیت AND

y	x_2	x_1
۰	۰	۰
۰	۱	۰
۰	۰	۱
۱	۱	۱

شکل ۲: نورون آستانه‌ای AND



محاسبات به صورت زیر است:

• ورودی (۰,۰):

$$z = 1 \times 0 + 1 \times 0 + (-1,5) = -1,5 < 0 \rightarrow y = 0$$

• ورودی (۱,۰):

$$z = 1 \times 0 + 1 \times 1 + (-1,5) = -0,5 < 0 \rightarrow y = 0$$

• ورودی (۰,۱):

$$z = 1 \times 1 + 1 \times 0 + (-1,5) = -0,5 < 0 \rightarrow y = 0$$

• ورودی (۱,۱):

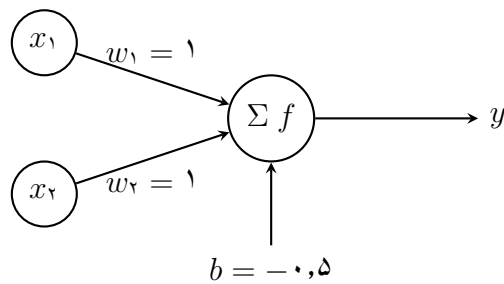
$$z = 1 \times 1 + 1 \times 1 + (-1,5) = 0,5 \geq 0 \rightarrow y = 1$$

گیت OR ۳-۱-۱

جدول ۳: جدول درستی گیت OR

y	x_2	x_1
۰	۰	۰
۱	۱	۰
۱	۰	۱
۱	۱	۱

شکل ۳: نورون آستانه‌ای OR



محاسبات به صورت زیر است:

• ورودی (۰,۰):

$$z = 1 \times 0 + 1 \times 0 + (-1,5) = -1,5 < 0 \rightarrow y = 0$$

• ورودی (۱،۰):

$$z = 1 \times 0 + 1 \times 1 + (-0.5) = 0.5 \geq 0 \rightarrow y = 1$$

• ورودی (۰،۱):

$$z = 1 \times 1 + 1 \times 0 + (-0.5) = 0.5 \geq 0 \rightarrow y = 1$$

• ورودی (۱،۱):

$$z = 1 \times 1 + 1 \times 1 + (-0.5) = 1.5 \geq 0 \rightarrow y = 1$$

۱-۱-۴ چرا یک نورون آستانه‌ای فقط می‌تواند توابع جداسازی‌پذیر خطی را نمایش دهد؟

یک نورون آستانه‌ای خروجی خود را بر اساس جمع وزن‌دار ورودی‌ها و یک بایاس محاسبه می‌کند. رابطه‌ی ریاضی آن به‌صورت $y = f(\mathbf{w} \cdot \mathbf{x} + b)$ است، که در آن $\mathbf{w}$ بردار وزن‌ها، $\mathbf{x}$ بردار ورودی‌ها، و b بایاس است. تابع f در این نوع نورون معمولاً یک تابع آستانه‌ای است که فقط دو مقدار ۰ یا ۱ تولید می‌کند. اگر مقدار $\mathbf{w} \cdot \mathbf{x} + b = 0$ را در نظر بگیریم، این رابطه معادله‌ی یک خط مستقیم در فضای دوبعدی (و به‌طور کلی یک ابرصفحه (Hyperplane) در فضای چندبعدی) را تشکیل می‌دهد. این خط مرز تصمیم نورون است، زیرا نقاطی که در یک سمت آن قرار دارند خروجی ۱، و نقاط سمت دیگر خروجی ۰ تولید می‌کنند. از آنجا که این مرز با یک معادله‌ی خطی توصیف می‌شود، همیشه شکل آن خطی است و نمی‌تواند خمیده یا منحنی باشد. در واقع، نورون آستانه‌ای هیچ سازوکار ریاضی برای ایجاد خمیدگی یا تغییر شکل در مرز تصمیم ندارد، زیرا تابع آن تنها بر اساس ترکیب خطی ورودی‌ها تعریف شده است. به همین دلیل، چنین نورونی فقط قادر به جدا کردن داده‌هایی است که در فضای ورودی با یک خط یا صفحه‌ی مستقیم از هم جدا می‌شوند.

۲-۱ Majority و Parity

در این بخش نخست توضیح می‌دهیم چرا تابع Parity با ورودی سه‌تایی را نمی‌توان با یک نوروں آستانه‌ای پیاده‌سازی کرد. سپس کوچکترین شبکه دولایه آستانه‌ای خود را تعریف خواهیم کرد و نشان خواهیم داد که پاسخ صحیح را تولید می‌کند. در ادامه همین روند را برای تابع Majority پی خواهیم گرفت و تحلیل خواهیم کرد که چرا این تابع با یک نوروں آستانه‌ای پیاده‌سازی می‌شود.

۱-۲-۱ توجیه هندسی عدم امکان نمایش با یک نوروں

تابع Parity با سه ورودی، زمانی مقدار خروجی ۱ تولید می‌کند که تعداد ورودی‌های برابر با ۱ فرد باشد (یعنی یک یا سه تا از ورودی‌ها برابر ۱ باشند) و در غیر این صورت مقدار خروجی ۰ است. اگر فضای ورودی‌ها را به صورت سه‌بعدی در نظر بگیریم، هر ورودی ممکن به صورت یک نقطه با مختصات (x_1, x_2, x_3) در این فضا نمایش داده می‌شود. از آنجا که هر ورودی می‌تواند صفر یا یک باشد، در مجموع هشت ترکیب ممکن داریم که دقیقاً رأس‌های یک مکعب واحد را تشکیل می‌دهند.

در این مکعب، نقاطی که خروجی تابع برابر با ۱ دارند (یعنی تعداد یک‌های فرد) عبارت‌اند از:

$$(0, 0, 1), (0, 1, 0), (1, 0, 0), (1, 1, 1)$$

و نقاطی که خروجی تابع برابر با ۰ دارند (تعداد یک‌های زوج) عبارت‌اند از:

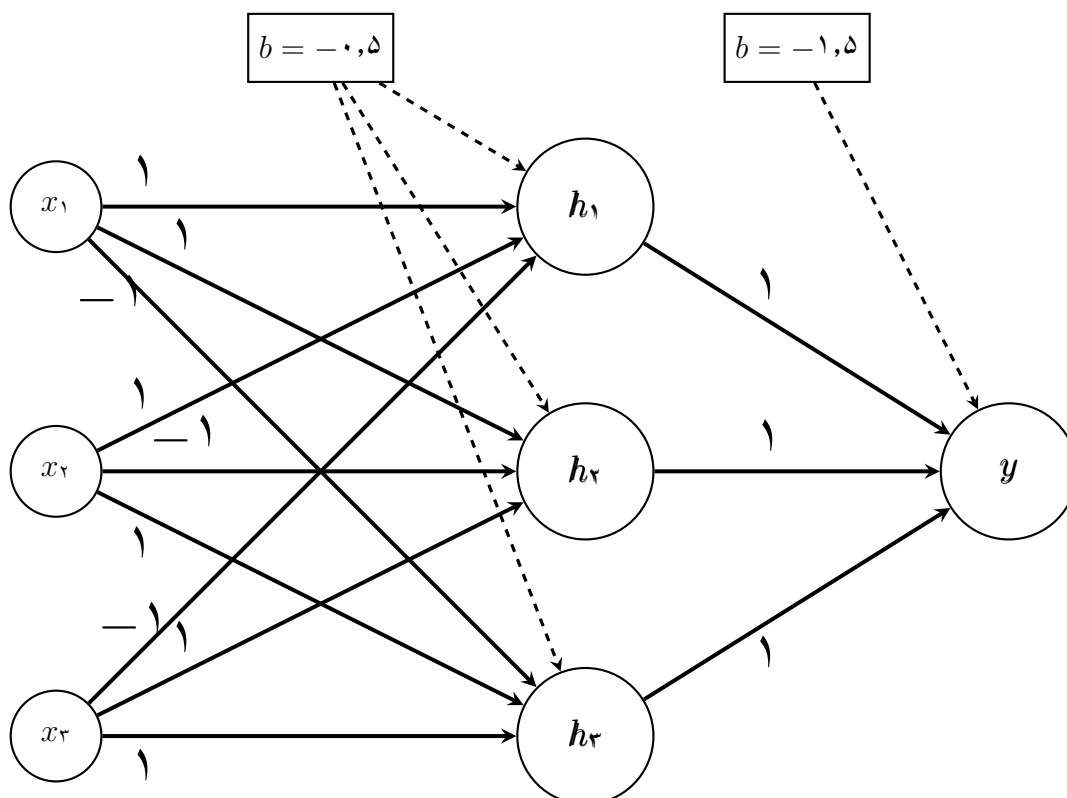
$$(0, 0, 0), (0, 1, 1), (1, 0, 1), (1, 1, 0)$$

اگر این نقاط را در فضای سه‌بعدی رسم کنیم، مشاهده می‌شود که نقاط با خروجی ۱ و نقاط با خروجی ۰ به صورت متناوب در کل مکعب پراکنده‌اند؛ به طوری که هر نقطه‌ای با خروجی ۱ توسط نقاطی با خروجی ۰ احاطه شده است و بالعکس.

بنابراین، تابع Parity سه‌ورودی غیرقابل جداسازی خطی است و یک نوروں آستانه‌ای به‌تنهایی نمی‌تواند آن را پیاده‌سازی کند. برای نمایش چنین تابعی باید از چندین نوروں آستانه‌ای یا یک شبکه چندلایه (مانند شبکه‌ی Madaline یا پرسپترون چندلایه) استفاده کرد تا ترکیب چند مرز خطی بتواند مرز تصمیم غیرخطی مورد نیاز

۲-۲-۱ طراحی شبکه برای تابع Parity

شکل ۴: شبکه نرون آستانه‌ای برای تابع Parity



حال نشان می‌دهیم که چرا معماری پیشنهادی درست است. با توجه به مقارن بودن وزن‌های لایه اول، روی تعداد یک‌های ورودی، حالت‌بندی می‌کنیم و هر حالت را بررسی می‌کنیم.

- **ورودی $(0, 0, 0)$:** اگر ورودی $(0, 0, 0)$ باشد، تمام h_i ها برابر -0.5 و در نتیجه صفر خواهد شد. در لایه دوم مقدار y برابر $b = -1.5$ خواهد شد که در نتیجه خروجی صفر را به ما می‌دهد.
- **ورودی $(1, 0, 0)$:** اگر ورودی $(1, 0, 0)$ باشد، h برابر $(0.5, -0.5, -0.5)$ می‌شود که بعد از گذر از آستانه برابر خواهد شد با $(0, 1, 1)$ در لایه دوم مقدار y برابر 0.5 خواهد شد که در نتیجه خروجی یک را به ما می‌دهد.

۱ نورون MacCulloch-Pitts

• ورودی $(0, 1, 1)$: اگر ورودی $(1, 1, 0)$ باشد، h برابر $(1, 0.5, -0.5)$ می‌شود که بعد از گذر از آستانه برابر خواهد شد با $(0, 0, 1)$ در لایه دوم مقدار y برابر ۱- خواهد شد که در نتیجه خروجی صفر را به ما می‌دهد.

• ورودی $(1, 1, 1)$: اگر ورودی $(1, 1, 1)$ باشد، تمام h_i ها برابر ۱, ۰, ۱ و در نتیجه یک خواهد شد. در لایه دوم مقدار y برابر ۳ خواهد شد که در نتیجه خروجی یک را به ما می‌دهد.

می‌توان به طور خلاصه‌تر بررسی درستی را در جدول زیر مشاهده نمود.

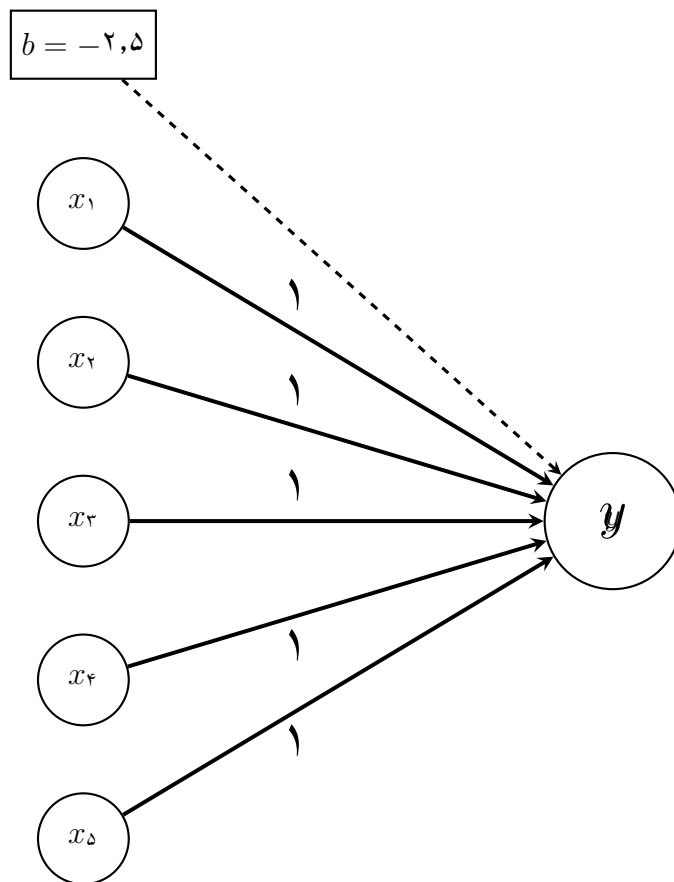
جدول ۴: خروجی شبکه برای ورودی‌های تابع Parity

x_1	x_2	x_3	h_1	h_2	h_3	$\sum h_i$	y	موردانتظار
۰	۰	۰	۰	۰	۰	۰	۰	۰
۰	۰	۱	۰	۱	۱	۲	۱	۱
۰	۱	۰	۱	۰	۱	۲	۱	۱
۰	۱	۱	۰	۰	۱	۱	۰	۰
۱	۰	۰	۱	۱	۰	۲	۱	۱
۱	۰	۱	۰	۱	۰	۱	۰	۰
۱	۱	۰	۱	۰	۰	۱	۰	۰
۱	۱	۱	۱	۱	۱	۳	۱	۱

۳-۲-۱ طراحی شبکه برای تابع Parity

با توجه به اینکه تابع Majority ورودی‌های باینری دارد و خروجی آن مقدار اکثریت ورودی‌هاست؛ پس می‌توان با یک جمع ساده بین ورودی‌ها برای این تابع یک مدل خطی پیشنهاد نمود. بنابراین برای طراحی شبکه نورون آستانه‌ای، نیازی به لایه پنهان نداریم و با یک نورون آستانه‌ای، این تابع مدل می‌شود.

شکل ۵: نرون آستانه‌ای برای تابع Majority



۲ Madaline و Adaline

۱-۲

۱-۱-۲ مشتق‌گیری و استخراج روابط، SSE LMS و منظم‌سازی Ridge

در این بخش، نمادها به صورت زیر تعریف می‌شوند:

• $x_i \in \mathbb{R}^d$ — بردار ورودی برای نمونه‌ی i ام.

• $t_i \in \{-1, +1\}$ — برجسب هدف برای نمونه‌ی i ام.

• $w \in \mathbb{R}^d$ — بردار وزن (در صورت لزوم، بایاس را می‌توان با افزودن یک مقدار ثابت به x در نظر گرفت).

• پیش‌بینی مدل: $\hat{y}_i = w^\top x_i$.

• ماتریس داده‌ها: X ماتریس $N \times d$ است که سطر i ام آن برابر $x_i^\top$ می‌باشد.

تابع مجموع مربعات خطا (SSE) Errors Squared of Sum به صورت زیر تعریف می‌شود:

$$E(w) = \frac{1}{2} \sum_{i=1}^N (t_i - w^\top x_i)^2$$

گرادیان

مشتق تابع $E(w)$ نسبت به w برای هر نمونه برابر است با:

$$\frac{\partial}{\partial w} \frac{1}{2} (t_i - w^\top x_i)^2 = -(t_i - w^\top x_i) x_i.$$

با جمع کردن روی تمام نمونه‌ها خواهیم داشت:

$$\nabla_w E(w) = - \sum_{i=1}^N (t_i - w^\top x_i) x_i.$$

بهروزرسانی LMS به صورت Batch (نزول گرادین)

قاعده‌ی بهروزرسانی نزول گرادین با نرخ یادگیری $\eta > 0$ به صورت زیر است:

$$w \leftarrow w - \eta \nabla_w E(w) = w + \eta \sum_{i=1}^N (t_i - w^\top x_i) x_i.$$

- این رابطه همان بهروزرسانی LMS دسته‌ای (Batch LMS) است که در هر تکرار از تمام نمونه‌ها استفاده می‌کند.
- اگر بهروزرسانی در پایان هر گذر (epoch) انجام شود، یادگیری به صورت دسته‌ای (batch) است.
- برای مقادیر کوچک η ، تکرار مداوم این بهروزرسانی وزن‌ها را به سمت جواب کمترین مربعات سوق می‌دهد.

ارتباط با معادلات نرمال (Normal Equations)

با برابر قرار دادن گرادین با صفر داریم:

$$-\sum_{i=1}^N (t_i - w^\top x_i) x_i = 0 \implies \sum_{i=1}^N x_i x_i^\top w = \sum_{i=1}^N x_i t_i.$$

به فرم ماتریسی:

$$X^\top X w = X^\top t.$$

اگر $X^\top X$ وارون پذیر باشد، آنگاه:

$$w^* = (X^\top X)^{-1} X^\top t.$$

بنابر این، بهروزرسانی دسته‌ای LMS به صورت تدریجی وزن‌ها را به سمت جواب بهینه‌ی کمترین مربعات هدایت می‌کند.

LMS آنلین (گرادین کاهش تصادفی)

زمانی که تنها از یک نمونه (x_n, t_n) در هر بار استفاده شود، قاعده‌ی Widrow-Hoff یا LMS آنلین

به‌دست می‌آید:

$$w \leftarrow w + \eta(t_n - w^\top x_n)x_n.$$

این روش همان **گرادیان کاهشی تصادفی (SGD)** است. در حالت میانگین، تقریب خوبی از گرادیان دسته‌ای ارائه می‌دهد و برای مقادیر کوچک η ، به مینیمم همگرا می‌شود.

چرا افزودن منظم‌سازی L2 منجر به رگرسیون Ridge می‌شود

هدف در SSE ، یافتن بردار وزن w است که **جمع مربعات خطاها (SSE)** را کمینه کند:

$$E(w) = \frac{1}{N} \sum_{i=1}^N (t_i - w^\top x_i)^2.$$

در قالب ماتریسی، که در آن X یک ماتریس $N \times d$ و t یک بردار $N \times 1$ است:

$$E(w) = \frac{1}{N} \|t - Xw\|^2.$$

گرادیان نسبت به w برابر است با:

$$\nabla_w E = -X^\top (t - Xw).$$

با برابر صفر قرار دادن گرادیان، **معادلات نرمال** به‌دست می‌آید:

$$X^\top Xw = X^\top t,$$

و بنابراین رامحل بسته:

$$w = (X^\top X)^{-1} X^\top t.$$

حال، **منظم‌سازی L2** جمله‌ای افزوده به تابع هزینه اضافه می‌کند که متناسب با اندازه‌ی مربعی وزن‌هاست:

$$E_{\text{ridge}}(w) = \frac{1}{N} \|t - Xw\|^2 + \frac{\lambda}{N} \|w\|^2,$$

که در آن $\lambda \geq 0$ میزان جریمه‌ی وزن‌های بزرگ را کنترل می‌کند.

این روش به‌نام رگرسیون **Ridge** یا **منظم‌سازی Tikhonov** شناخته می‌شود.

با مشتق‌گیری نسبت به w :

$$\nabla_w E_{\text{ridge}} = -X^\top(t - Xw) + \lambda w.$$

در مقایسه با گرادینان بدون منظم‌سازی $-X^\top(t - Xw)$ ، جمله‌ی اضافی λw باعث می‌شود وزن‌ها به سمت صفر کشیده شوند.

با قرار دادن $\nabla_w E_{\text{ridge}} = 0$:

$$-X^\top(t - Xw) + \lambda w = 0.$$

ساده می‌شود:

$$(X^\top X + \lambda I)w = X^\top t.$$

در نتیجه راحل نهایی:

$$w_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top t.$$

جدول ۵: آماره های داده ها قبل و بعد از نرمال سازی

Feature	(Before) Mean	(Before) Std	MinMax) (After Mean	MinMax) (After Std
radius mean	14.1044	3.6181	0.3141	0.1772
texture mean	19.2193	4.2659	0.3216	0.1443
perimeter mean	91.8105	24.9448	0.3122	0.1774
area mean	654.6794	363.4432	0.2078	0.1559
smoothness mean	0.0960	0.0139	0.4831	0.1550
compactness mean	0.1041	0.0532	0.2600	0.1632
concavity mean	0.0887	0.0783	0.2362	0.2085
points concave mean	0.0490	0.0393	0.2563	0.2057
symmetry mean	0.1801	0.0270	0.3743	0.1362
dimension fractal mean	0.0627	0.0071	0.2691	0.1491
error radius	0.4013	0.2874	0.1050	0.1041
error texture	1.2110	0.5428	0.2406	0.1535
error perimeter	2.8505	2.1208	0.0986	0.0999
error area	40.3141	48.4838	0.0626	0.0906
error smoothness	0.0071	0.0032	0.1558	0.1108
error compactness	0.0255	0.0173	0.1743	0.1300
error concavity	0.0317	0.0268	0.1044	0.0883
error points concave	0.0119	0.0061	0.2900	0.1501
error symmetry	0.0206	0.0084	0.1597	0.1209
error dimension fractal	0.0037	0.0023	0.1353	0.1101
radius worst	16.2185	4.8950	0.2756	0.1789
texture worst	25.5742	6.0471	0.3857	0.1721
perimeter worst	106.9587	34.0380	0.2667	0.1730
area worst	876.7646	576.9036	0.1621	0.1431
smoothness worst	0.1320	0.0228	0.3588	0.1610
compactness worst	0.2545	0.1571	0.2436	0.1738
concavity worst	0.2709	0.2049	0.2163	0.1636
points concave worst	0.1146	0.0664	0.3948	0.2288
symmetry worst	0.2893	0.0609	0.2617	0.1201
dimension fractal worst	0.0839	0.0178	0.2432	0.1514

جدول ۶: توازن کلاس ها

Class	Count	Percentage
malignant	148	37.19
benign	250	62.81

بهترین مدل Adaline بر اساس دقت روی مجموعه تست

بهترین مدل آموزش دیده با پارامترهای زیر انتخاب شد:

- نرخ یادگیری: $\eta = 0.0005$
- ضریب منظم‌سازی L2: 0.1
- دقت روی مجموعه تست: 0.9240

ارزیابی روی مجموعه تست

ماتریس درهم‌ریختگی:

جدول ۷: ماتریس درهم‌ریختگی مدل Adaline روی مجموعه تست

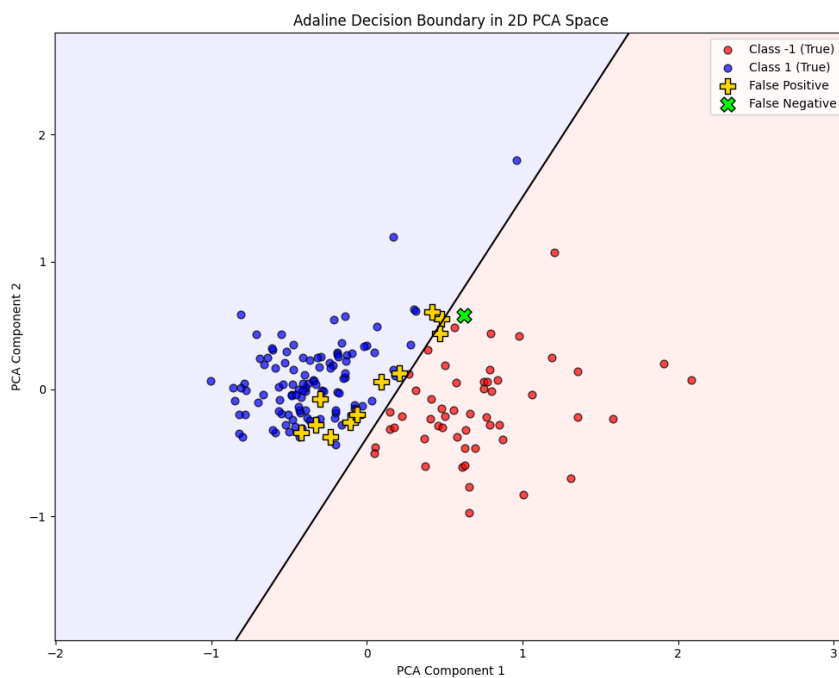
پیش‌بینی منفی (benign) پیش‌بینی مثبت (malignant)		
12	52	واقعا منفی (benign)
106	1	واقعا مثبت (malignant)

معیارهای عملکرد:

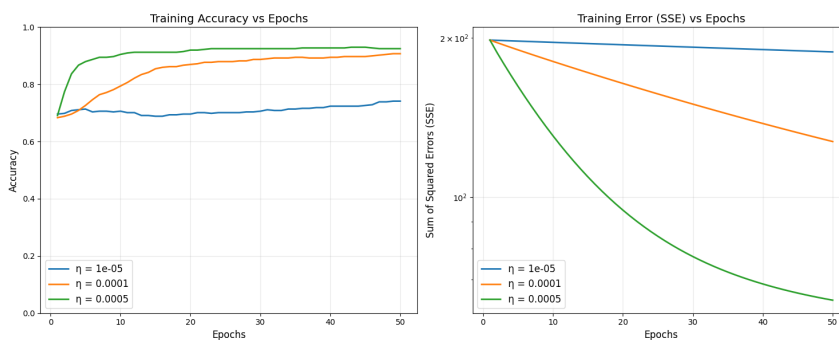
جدول ۸: معیارهای سنجش مدل Adaline روی داده آزمون

Value	Metric
0.9240	Accuracy
0.8983	Precision
0.9907	Recall
0.9422	Score 'F
0.9740	ROC-AUC

Madaline و Adaline ۲



شکل ۶: pca



شکل ۷: نمودار کاهش خطای Adaline

۴-۱-۲ مقایسه با مدل‌های پایه

جدول ۹: مقایسه‌ی عملکرد مدل‌ها بر اساس معیارهای مختلف

مدل	دقت (Accuracy)	دقت مثبت (Precision)	بازخوانی (Recall)	F _۱	AUC ROC
Adaline	0.9415	0.9292	0.9813	0.9545	0.9867
LDA	0.9532	0.9459	0.9813	0.9633	0.9855
رگرسیون لجستیک	0.9591	0.9717	0.9626	0.9671	0.9956

عملکرد **Adaline** بهتر است زمانی که:

۱. داده‌ها به‌صورت کامل خطی جداساز باشند و حاشیه‌ی جداسازی (margin) بزرگ باشد.
۲. سرعت محاسبات و سادگی الگوریتم اهمیت زیادی داشته باشد (بمدلیل استفاده از گرادیان نزولی ساده).
۳. ویژگی‌ها از پیش به‌خوبی نرمال‌سازی یا مقیاس‌بندی شده باشند.

عملکرد **Adaline** ضعیف‌تر است زمانی که:

۱. همپوشانی (overlap) بین کلاس‌ها زیاد باشد و مرز تصمیم واضحی وجود نداشته باشد.
 ۲. دقت بالا در پیش‌بینی کلاس مثبت (Precision) مورد نیاز باشد؛ زیرا در این معیار عملکرد **Adaline** پایین‌تر است.
 ۳. تخمین‌های احتمالاتی دقیق برای تصمیم‌گیری لازم باشد (**Adaline**) فقط خروجی عددی خطی دارد و احتمال تولید نمی‌کند).
 ۴. توزیع ویژگی‌ها با فرض نرمال بودن (در **LDA** سازگار نباشد یا داده‌ها نیاز به مرزهای تصمیم غیرخطی داشته باشند).
- به‌طور کلی، مدل‌های احتمالاتی مانند **رگرسیون لجستیک** و **LDA** به دلیل ماهیت مبتنی بر تابع درست‌نمایی (Likelihood-based) و کالیبراسیون بهتر خروجی‌ها، در معیارهای ترکیبی (مانند F_1 و ROC AUC) عملکرد دقیق‌تر و پایدارتری نسبت به **Adaline** دارند.

الگوریتم ۱ شبکه سه‌لایه Adaline (قانون II، نسخه غیرخطی)

- ورودی: داده‌های آموزش $(X_{\text{train}}, y_{\text{train}})$ ، داده تست اختیاری $(X_{\text{test}}, y_{\text{test}})$ ، نرخ یادگیری η ، ضریب λ ، تعداد نورون‌های پنهان H ، بیشینه‌ی دوره‌ها E_{max}
- خروجی: وزن‌های نهایی W_1, W_2, b_1, b_2 ، دقت و خطای آموزش/تست
- ۱: مقداردهی اولیه $W_1 \in \mathbb{R}^{d \times H}$ ، $W_2 \in \mathbb{R}^H$ (Xavier)
 - ۲: مقداردهی اولیه $b_1 = 0$ ، $b_2 = 0$
 - ۳: تعریف تابع فعال‌سازی $f(x) = \tanh(x)$ و مشتق آن $f'(x) = 1 - \tanh^2(x)$
 - ۴: برای epoch = ۱ تا E_{max} انجام بده
 - ۵: $n_updates \leftarrow 0$
 - ۶: برای هر نمونه آموزشی (x_i, y_i) انجام بده
 - ۷: $z_1 \leftarrow x_i \cdot W_1 + b_1$
 - ۸: $h \leftarrow f(z_1)$
 - ۹: $z_2 \leftarrow h \cdot W_2 + b_2$
 - ۱۰: $y \leftarrow f(z_2)$
 - ۱۱: $error \leftarrow y_i - y$
 - ۱۲: اگر $|error| > 0.1$ یا $\text{sign}(y) \neq y_i$ آنگاه
 - ۱۳: اعمال منظم‌سازی λ : $W_1 \leftarrow W_1(1 - \eta\lambda)$ ، $W_2 \leftarrow W_2(1 - \eta\lambda)$
 - ۱۴: $\delta_2 \leftarrow error \cdot f'(z_2)$
 - ۱۵: $\delta_1 \leftarrow \delta_2 \cdot W_2 \cdot f'(z_1)$
 - ۱۶: به‌روزرسانی وزن‌های لایه خروجی: $W_2 \leftarrow W_2 + \eta\delta_2 h$ ، $b_2 \leftarrow b_2 + \eta\delta_2$
 - ۱۷: به‌روزرسانی وزن‌های لایه پنهان: $W_1 \leftarrow W_1 + \eta x_i^T \delta_1$ ، $b_1 \leftarrow b_1 + \eta\delta_1$
 - ۱۸: $n_updates \leftarrow n_updates + 1$
 - ۱۹: پایان اگر
 - ۲۰: پایان برای
 - ۲۱: محاسبه دقت و خطای آموزش
 - ۲۲: اگر داده تست موجود باشد آنگاه
 - ۲۳: محاسبه دقت و خطای تست
 - ۲۴: پایان اگر
 - ۲۵: اگر $n_updates = 0$ آنگاه
 - ۲۶: خاتمه زودهنگام (همگرا شد)
 - ۲۷: پایان اگر
 - ۲۸: پایان برای

جدول ۱۰: کارایی بهترین مدل روی داده‌های ماه و دایره

Size Hidden	ROC-AUC	F1	Recall	Precision	Accuracy	Dataset
۵	0.9607	0.8994	0.8636	0.9383	0.9056	Moons
۵	0.9975	0.9829	0.9773	0.9885	0.9833	Circles

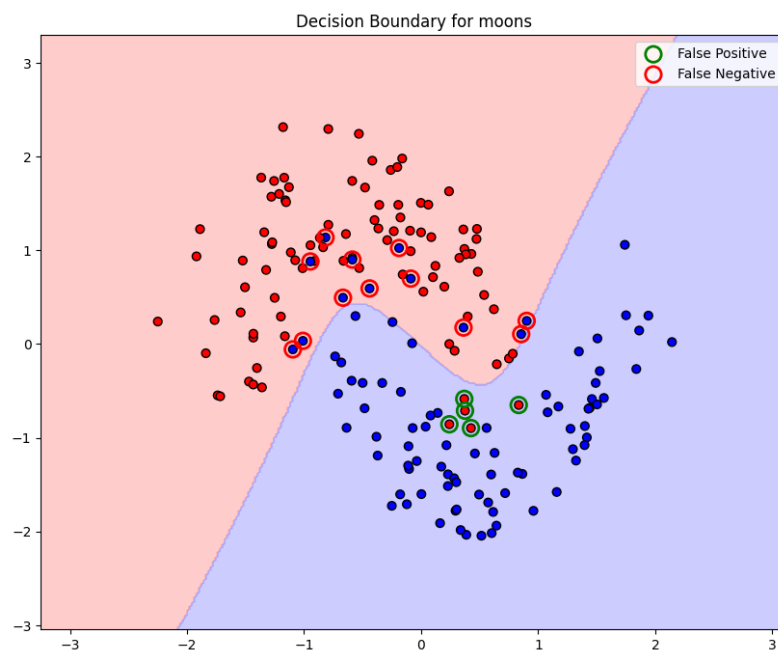
جدول ۱۱: ماتریس درهم‌ریختگی برای داده ماه

۱+ Class	۱- Class	Actual / Predicted
5	87	۱- Class
76	12	۱+ Class

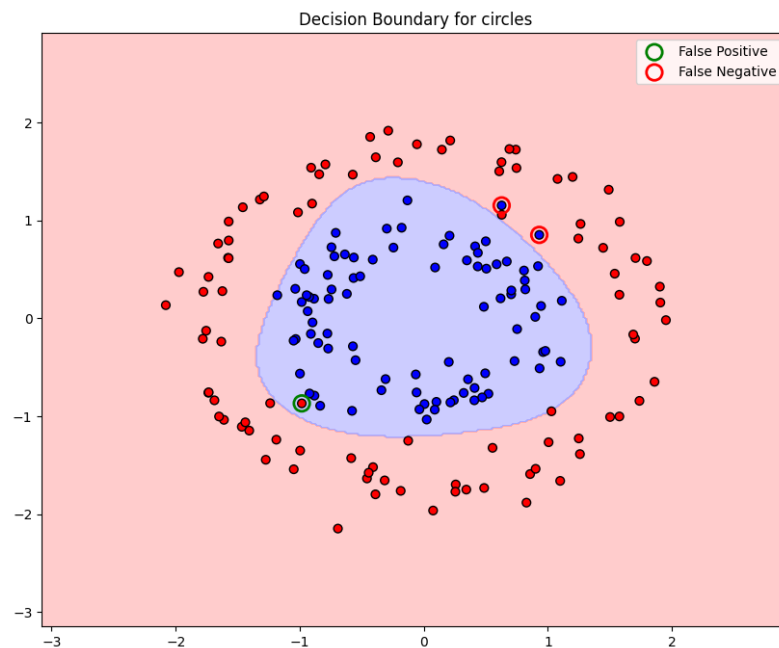
جدول ۱۲: ماتریس درهم‌ریختگی برای داده دایره

۱+ Class	۱- Class	Actual / Predicted
1	91	۱- Class
86	2	۱+ Class

مرز تصمیم

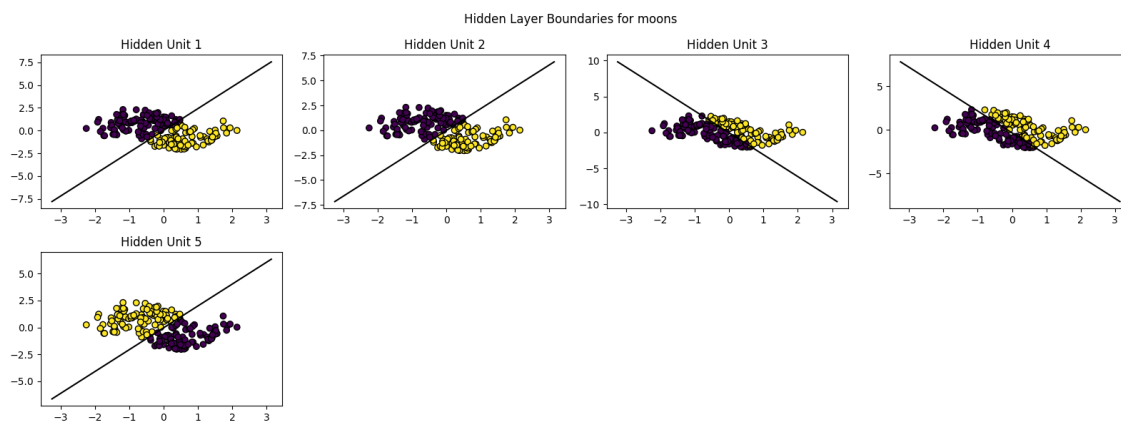


شکل ۸: مرز تصمیم داده ماه

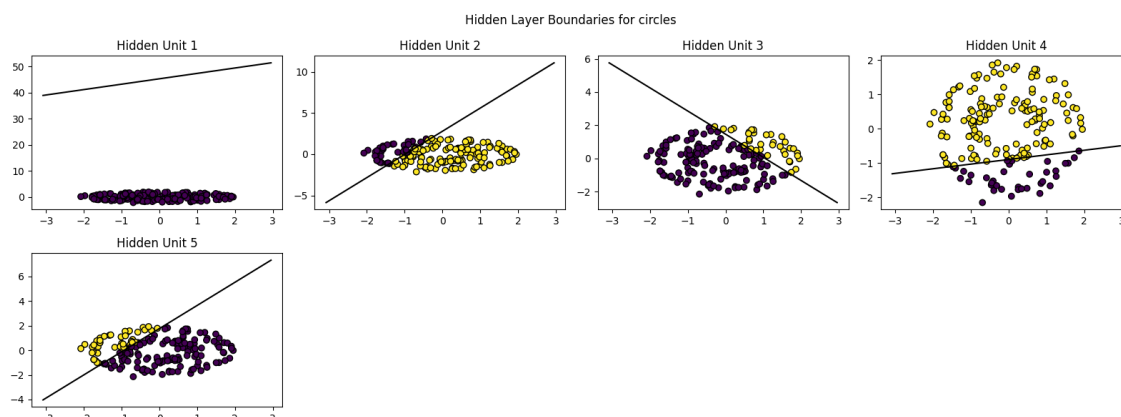


شکل ۹: مرز تصمیم داده دایره

لایه‌های پنهان



شکل ۱۰: لایه‌های پنهان داده ماه



شکل ۱۱: لایه‌های پنهان داده دایره

در شبکه‌ی سه‌لایه‌ی آدالین (Three-Layer Adaline)، هر نورون در لایه‌ی پنهان یک مرز تصمیم خطی ایجاد می‌کند. خروجی هر نورون پنهان برابر با تابعی غیرخطی (مانند $\tanh$) از ترکیب خطی ورودی‌ها است. این خروجی‌ها سپس به‌عنوان ورودی به لایه‌ی خروجی داده می‌شوند. به این ترتیب، چندین مرز خطی که توسط نورون‌های پنهان ساخته شده‌اند، با هم ترکیب می‌شوند تا یک مرز تصمیم نهایی و غیرخطی ایجاد کنند. این ترکیب به مدل امکان می‌دهد داده‌هایی را که به‌صورت خطی قابل جداسازی نیستند (مانند داده‌های moons یا circles)، به‌خوبی تفکیک کند. سپس خروجی هر نورون با یک تابع غیرخطی (مثل $\tanh$) عبور داده می‌شود تا مقدار آن بین -1 و 1 محدود شود:

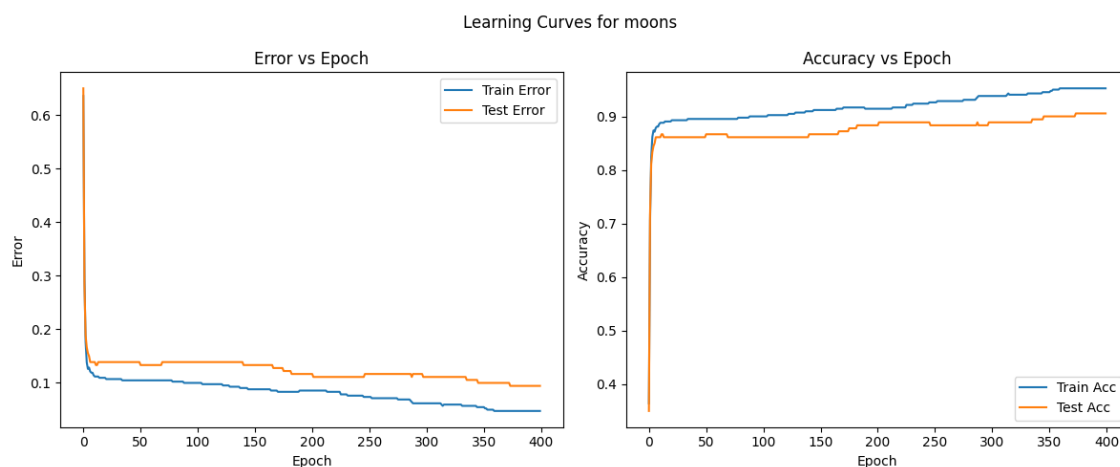
$$h_i = \tanh(z_i)$$

لایه‌ی خروجی این خروجی‌های غیرخطی را به صورت ترکیب خطی با ضرایب v_i جمع می‌کند:

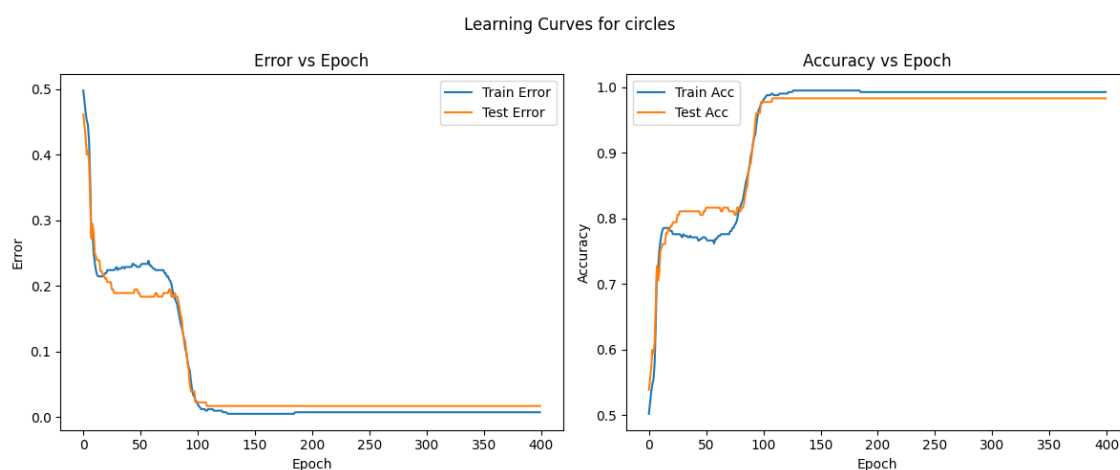
$$y = \tanh\left(\sum_i v_i h_i + b_y\right)$$

در نتیجه، مرز نهایی تصمیم حاصل از ترکیب غیرخطی چندین صفحه‌ی خطی است.

نمودار خطا



شکل ۱۲: نمودار کاهش خطای داده ماه



شکل ۱۳: نمودار کاهش خطای داده دایره

۳-۲-۲ مطالعه ابرپارامترها

ارتباط بیش‌برازش و کم‌برازش با مرز تصمیم:

در مسئله‌های طبقه‌بندی، مرز تصمیم سطحی است که داده‌های متعلق به کلاس‌های مختلف را از هم جدا می‌کند. پیچیدگی این مرز بیانگر توانایی مدل در یادگیری الگوهای داده است.

اگر مدل بیش از حد ساده باشد (مانند یک مرز خطی برای داده‌های غیرخطی)، مرز تصمیم نمی‌تواند شکل واقعی داده‌ها را دنبال کند و مدل دچار کم‌برازش (Underfitting) می‌شود. در این حالت، مرز تصمیم صاف و

۲ Adaline و Madaline

Seed	Dataset	Accuracy	Precision	Recall	F ₁ -score	ROC AUC	Hidden LR, λ_L
۰	Moons	۰.۹۰۵۶	۰.۹۳۸۳	۰.۸۶۳۶	۰.۸۹۹۴	۰.۹۶۰۷	(5, 0.001, 0)
۱	Moons	۰.۹۲۷۸	۰.۹۵۷۴	۰.۹۰۹۱	۰.۹۳۲۶	۰.۹۸۷۳	(3, 0.001, 0)
۲	Moons	۰.۹۲۲۲	۰.۹۰۶۳	۰.۹۴۵۷	۰.۹۲۵۵	۰.۹۸۱۳	(5, 0.001, 0.001)
۰	Circles	۰.۹۸۳۳	۰.۹۸۸۵	۰.۹۷۷۳	۰.۹۸۲۹	۰.۹۹۷۵	(5, 0.001, 0)
۱	Circles	۰.۹۹۴۴	۰.۹۹۰۰	۱.۰۰۰۰	۰.۹۹۵۰	۰.۹۹۹۵	(7, 0.001, 0)
۲	Circles	۱.۰۰۰۰	۱.۰۰۰۰	۱.۰۰۰۰	۱.۰۰۰۰	۱.۰۰۰۰	(10, 0.001, 0)

جدول ۱۳: کارایی مدل‌ها بر اساس seed روی داده‌های ماه و دایره

ساده است و داده‌ها را به‌خوبی تفکیک نمی‌کند.

در مقابل، اگر مدل بیش از حد پیچیده باشد (مثلاً دارای لایه‌ها یا نورون‌های زیاد در شبکه‌ی عصبی)، مرز تصمیم بسیار پیچ‌خورده می‌شود و حتی نویز داده را نیز یاد می‌گیرد؛ در نتیجه مدل دچار بیش‌برازش (Overfit-ting) می‌شود و عملکرد آن روی داده‌های جدید کاهش می‌یابد.

به‌طور خلاصه، پیچیدگی مرز تصمیم مستقیماً با میزان برآزش مدل در ارتباط است: مرز ساده → کم‌برآزش، مرز بسیار پیچیده → بیش‌برآزش. هدف، یافتن تعادلی میان این دو حالت است تا مرز تصمیم هم دقیق و هم قابل‌عموم‌سازی باشد.

۳ رگرشن با استفاده از mlp

در این بخش به پرسش‌های مربوط به سوال رگرشن با استفاده از mlp پاسخ می‌دهیم. نخست مجموعه دادگان Prices Housing California را نمایش و پیش‌پردازهای خواسته شده را انجام می‌دهیم. سپس دادگان را روی مدل پیشنهاد شده آموزش می‌دهیم و معیارهای سنجش مدل را گزارش می‌دهیم. در قدم بعد یه معماری پیچیده‌تر پیاده‌سازی می‌کنیم و نشان می‌دهیم که عملکرد مدل پیچیده‌تر بهتر از مدل نخستین است. در نهایت این بخش را با تحلیل چند نمودار به پایان خواهیم برد.

۱-۳ پیش‌پردازش داده‌ها

قبل از آموزش هر مدل شبکه عصبی، نیاز به پیش‌پرداز داده‌ها داریم.

۱-۱-۳ مدیریت values missing

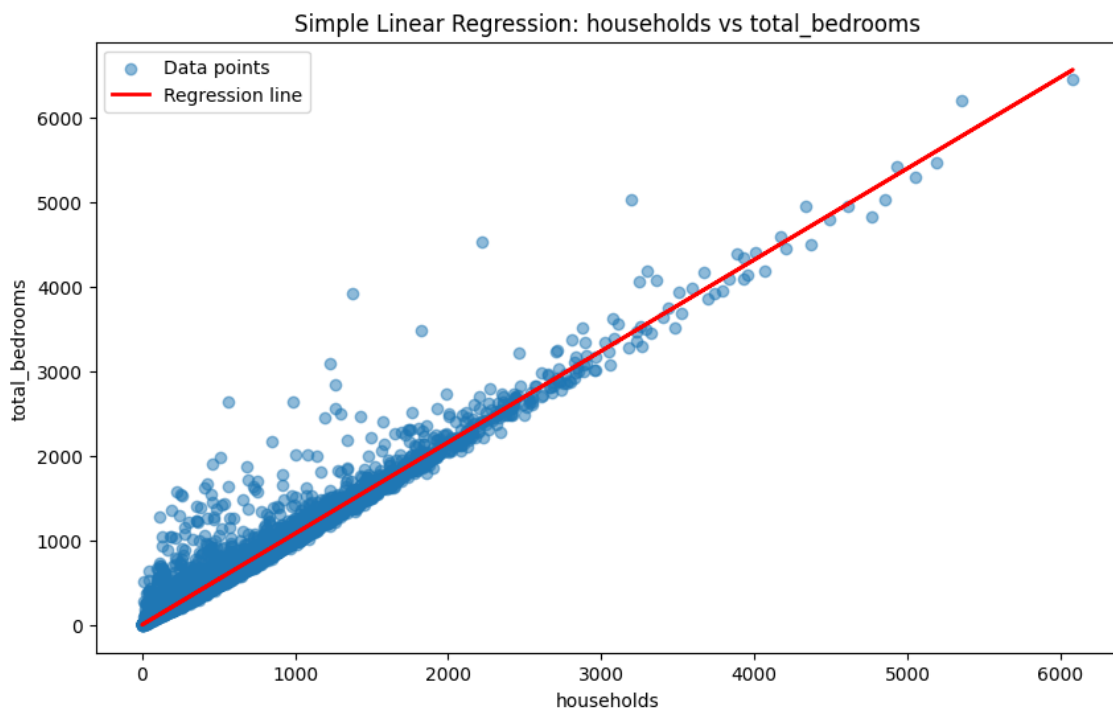
در ابتدا نگاه می‌کنیم که کدام ستون یا سطرها دارای مقادیر گم‌شده است. با توجه به دیتاست تنها ستون total bedrooms دارای ۲۰۷ داده گم‌شده است. برای جانهی این مقادیر گم‌شده با توجه به عددی بودن ستون، استراتژی‌های زیر موجود است:

- **حذف سطر دارای مقدار گم‌شده** با انجام این کار باعث حذف داده‌ها و اطلاعات و کاهش واریانس بین داده‌ها می‌شویم.

- **جانهی میانگین همان ستون** در این کار سطرهای را از دست نمی‌دهیم اما این کار باعث می‌شود با داده‌های ناواقعی مواجه باشیم. همچنین واریانس آن ستون نیز کم می‌شود.

- **پیش‌بینی مقدار گم‌شده با استفاده از سایر ستون‌ها** در این کار با برآزش یک مدل رگرسیونی ساده، تلاش می‌کنیم تا داده گم‌شده را با استفاده از سایر ستون‌هایی که در اختیار داریم حدس بزنیم.

ما روش سوم را برگزیدیم. نخست کوریلیشن بین ستون دارای مقدار گم‌شده با سایر ستون‌ها را محاسبه کردیم و دیدیم ستون bedroom total با ستون household حدود ۹۸ درصد کوریلیشن دارد. پس یک خط رگرسیونی ساده بین این دو ستون برآزش دادیم و مقدار گم‌شده را براساس معادله رگرسیونی محاسبه شده پیدا کردیم.



شکل ۱۴: کورلیشن بین دو ستون

دلیل برخورد با مقادیر گمشده

۱. جلوگیری از اختلال در تحلیل آماری: بسیاری از الگوریتم‌ها و محاسبات آماری نمی‌توانند با مقادیر تهی کار کنند و ممکن است منجر به خطا یا نتایج نادرست شوند.
۲. افزایش دقت مدل: اگر مقادیر گمشده بدون جایگزینی یا حذف مناسب باقی بمانند، مدل نمی‌تواند الگوی واقعی داده‌ها را یاد بگیرد و دقت آن کاهش می‌یابد.

۲-۱-۳ حذف کردن outliers از قسمت قیمت

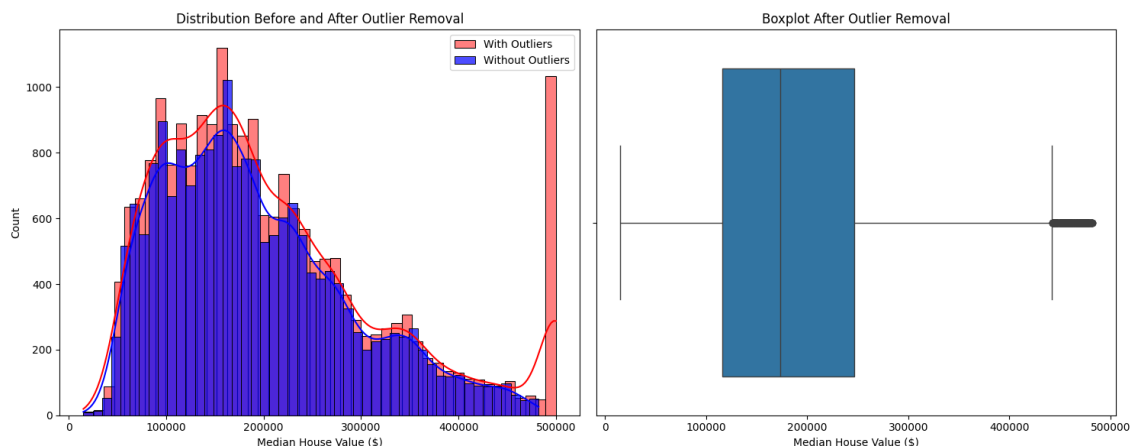
برای حذف داده‌های پرت، از بازه‌ی بین چارکی استفاده می‌کنیم. ابتدا چارک اول (Q_1) و چارک سوم (Q_3) داده‌ها را محاسبه می‌کنیم، سپس مقدار IQR به‌صورت زیر تعریف می‌شود:

$$IQR = Q_3 - Q_1$$

هر داده‌ای که بیرون از بازه‌ی زیر قرار گیرد، به عنوان داده‌ی پرت شناخته شده و حذف می‌شود:

$$[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$$

این روش باعث می‌شود اثر داده‌های پرت در تحلیل آماری و مدل‌سازی کاهش یابد.



شکل ۱۵: دیتاست قبل و بعد حذف outlier

دلیل حذف داده‌های پرت

۱. **افزایش دقت مدل:** داده‌های پرت ممکن است باعث شوند مدل آماری یا یادگیری ماشین نتواند الگوی اصلی داده‌ها را به درستی بیاموزد. با حذف آن‌ها، مدل روی داده‌های نماینده‌تر آموزش می‌بیند.
۲. **بهبود تفسیر نتایج:** وجود داده‌های پرت می‌تواند میانگین و واریانس را به شدت تغییر دهد و در نتیجه تحلیل‌های آماری را گمراه‌کننده کند. حذف آن‌ها باعث می‌شود نتایج آماری قابل اعتمادتر باشند.
۳. **کاهش خطاهای اندازه‌گیری یا ورود داده:** بسیاری از داده‌های پرت ناشی از خطای انسانی یا ابزار اندازه‌گیری هستند. حذف این داده‌ها موجب افزایش کیفیت داده و صحت نتایج می‌شود.

۳-۱-۳ تبدیل متغیرهای کیفی به یک مقدار قابل قبول

در بسیاری از مجموعه‌داده‌ها، برخی از ویژگی‌ها (متغیرها) به صورت Categorical یا دسته‌ای هستند؛ به عنوان مثال، ویژگی‌هایی مانند «ocean proximity» در دیتاست از آنجا که بیشتر الگوریتم‌های یادگیری ماشین

۳ رگرشن با استفاده از mlp

تنها با داده‌های عددی کار می‌کنند، لازم است این مقادیر دسته‌ای به شکل عددی تبدیل شوند. یکی از رایج‌ترین روش‌ها برای این کار، روش One-Hot Encoding است. در این روش، برای هر دسته از یک ویژگی، یک ستون جدید ایجاد می‌شود که مقدار آن صفر یا یک است. به این ترتیب، هر نمونه تنها در ستونی که نشان‌دهنده دسته مربوط به آن است مقدار «۱» می‌گیرد و در سایر ستون‌ها مقدار «۰» دارد. از آنجا که ستون ocean proximity پنج مقدار دارد، ۵ ستون جدید اضافه و خود ستون ocean proximity حذف می‌شود.

۲-۳ پیاده‌سازی شبکه عصبی پیشنهادشده

حال بعد از انجام پیش‌پردازش‌های لازم و افراز داده‌ها به سه مجموعه آموزش، ارزیابی و آزمون، مدل پیشنهادشده که به شرح زیر است را پیاده‌سازی می‌کنیم.

جدول ۱۴: معماری شبکه عصبی ساده

NN Module	Parameters
Linear	In = input_size, Out = ۸
ReLU (Activate Function) ۲)	
Linear ۳)	In = ۸, Out = ۱

این شبکه را با هابیرپارامترهای زیر روی دیتاست آموزش داده‌ایم.

جدول ۱۵: هابیرپارامترهای مدل پیشنهادشده

مقدار	هابیرپارامتر
۶۴	اندازه دسته (Batch Size)
۸۰٪ - ۱۰٪ - ۱۰٪	نسبت آموزش، ارزیابی، آزمون
Adam	بهینه‌ساز
۰,۱	نرخ یادگیری
MSE	تابع هزینه

۳-۳ معیارهای سنجش مدل‌های رگرسیونی

برای ارزیابی دقت و کارایی مدل‌های رگرسیونی، از معیارهای مختلفی استفاده می‌شود. در ادامه، چهار معیار پرکاربرد یعنی MSE، RMSE، MAE و R^2 معرفی و بررسی می‌شوند.

۱-۳-۳ میانگین مربعات خطا (Mean Squared Error - MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

در این معیار، میانگین مربع اختلاف بین مقادیر واقعی (y_i) و مقادیر پیش‌بینی‌شده ($\hat{y}_i$) محاسبه می‌شود. استفاده از توان دوم باعث می‌شود خطاهای بزرگ، تأثیر بیشتری در مقدار نهایی داشته باشند. هرچه مقدار MSE کمتر باشد، عملکرد مدل بهتر است. با این حال، واحد این معیار برابر با مجذور واحد متغیر هدف است و از این رو تفسیر مستقیم آن دشوار است.

مزایا:

- مجازات بیشتر برای خطاهای بزرگ.
- پیوسته و مشتق‌پذیر — مناسب برای الگوریتم‌های مبتنی بر گرادیان.

معایب:

- حساسیت زیاد به داده‌های پرت.
- واحد اندازه‌گیری متفاوت از متغیر هدف.

۲-۳-۳ ریشه میانگین مربعات خطا (Root Mean Squared Error - RMSE)

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE ریشه‌ی دوم MSE است و مقدار آن در همان واحد متغیر هدف بیان می‌شود. این معیار میزان متوسط انحراف پیش‌بینی‌ها از مقادیر واقعی را در واحد واقعی داده‌ها نشان می‌دهد.

مزایا:

- قابل تفسیر — دارای همان واحد متغیر هدف.
- حساس به خطاهای بزرگ، مناسب برای مدل‌های دقیق.

معایب:

- حساسیت بالا به داده‌های پرت (به دلیل توان دوم در محاسبه).

۳-۳-۳ میانگین قدر مطلق خطا (Mean Absolute Error - MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

این معیار میانگین قدر مطلق اختلاف بین مقادیر واقعی و پیش‌بینی‌شده را اندازه‌گیری می‌کند و تمام خطاها را به یک میزان در نظر می‌گیرد.

مزایا:

- مقاوم‌تر در برابر داده‌های پرت نسبت به MSE.

- قابل فهم و ساده در تفسیر.

معایب:

- خطاهای بزرگ را به اندازه MSE جریمه نمی‌کند.
- در نقطه صفر مشتق پذیر نیست (اما در عمل مشکلی ایجاد نمی‌کند).

۴-۳-۳ ضریب تعیین (R^2 Score)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

در این رابطه، $\bar{y}$ میانگین مقادیر واقعی است. این معیار نشان می‌دهد چه نسبتی از واریانس متغیر وابسته توسط مدل توضیح داده می‌شود.

- $R^2 = 1$: مدل به‌طور کامل مقادیر واقعی را پیش‌بینی کرده است.

- $R^2 = 0$: مدل هیچ بهتر از میانگین داده‌ها نیست.

- • $R^2 < 0$: مدل از پیش‌بینی میانگین نیز بدتر عمل کرده است.

مزایا:

- بدون واحد و قابل مقایسه بین مدل‌ها.
- نشان‌دهنده میزان توضیح‌دهندگی مدل نسبت به داده‌ها.

معایب:

- میزان خطای واقعی را نشان نمی‌دهد.
- در داده‌های غیرخطی با دارای نویز زیاد می‌تواند گمراهنده باشد.

جدول ۱۶: مقایسه معیارهای ارزیابی مدل

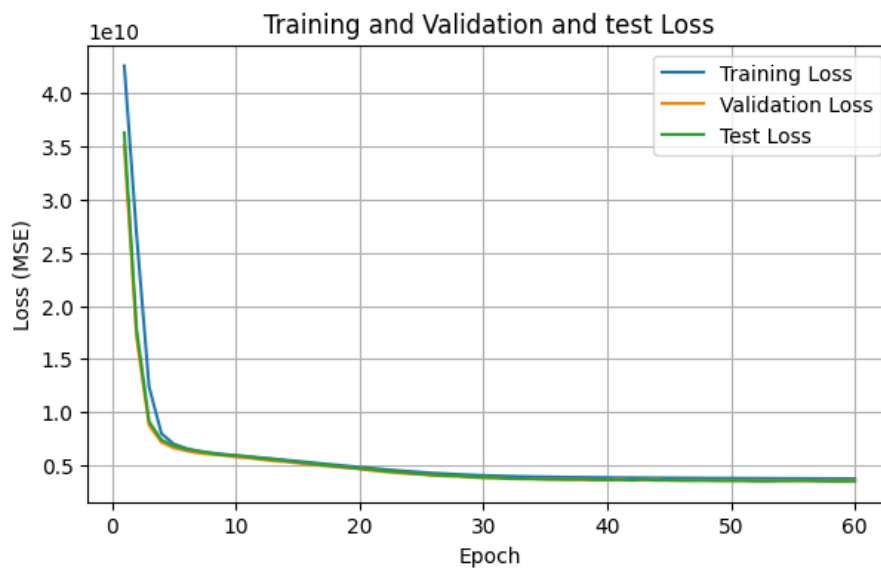
معیار	مقیاس	مجازات خطای بزرگ	حساسیت به پرت‌ها	قابلیت تفسیر	کاربرد اصلی
MSE	مجذور واحد هدف	بله (زیاد)	زیاد	متوسط	بهینه‌سازی مدل و تحلیل ریاضی
RMSE	همان واحد هدف	بله (زیاد)	زیاد	زیاد	تفسیر در مقیاس واقعی داده
MAE	همان واحد هدف	خیر	کم	بسیار زیاد	زمانی که همه خطاها به یک اندازه مهم‌اند
R^2	بدون واحد	خیر	متوسط	زیاد	ارزیابی قدرت توضیح‌دهندگی مدل

گزارش معیارهای سنجش روی مدل شبکه عصبی ساده

حال معیارهایی که توضیح دادیم را روی مدل شبکه عصبی پیشنهادشده محاسبه می‌کنیم.

جدول ۱۷: معیارهای سنجش روی شبکه عصبی ساده

R²	MAE	RMSE	MSE	Dataset
0.912.	0.469.45.89	6641.61.67	0.00.3729209776	Train
0.55.	2812.4376.	3242.0936.	0.00.3523648.00	Validation
0.185.	1680.44.65	0.352.0910.7	0.00.3493641472	Test



شکل ۱۶: نمودار کاهش خطا روی داده‌های آموزش، ارزیابی و آزمون روی شبکه عصبی ساده

۴-۳ پیاده‌سازی شبکه عصبی پیچیده‌تر

برای دستیابی به نتیجه بهتر، معماری پیچیده‌تر زیر را برگزیدیم.

جدول ۱۸: خلاصه معماری مدل پیچیده‌تر

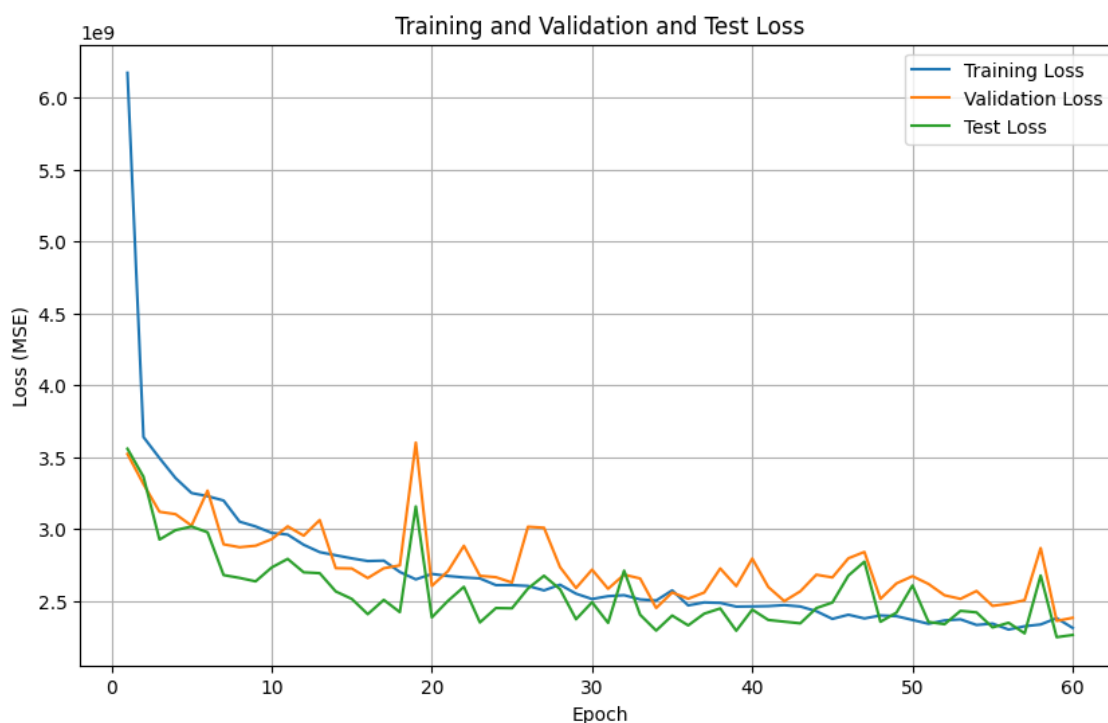
Layer (type)	Output Shape	Param #
Linear-۱	[۳۲، ۱، ۳۲]	۴۴۸
Linear-۲	[۳۲، ۱، ۶۴]	۲،۱۱۲
Linear-۳	[۳۲، ۱، ۱۶]	۱،۰۴۰
Linear-۴	[۳۲، ۱، ۱]	۱۷
Total params		۳،۶۱۷
Trainable params		۳،۶۱۷
Non-trainable params		۰

این شبکه را با هابپرپارامترهای زیر روی دیتاست آموزش داده‌ایم.

جدول ۱۹: هایپر پارامترهای مدل پیچیدمتر

هایپر پارامتر	مقدار
اندازه دسته (Batch Size)	۳۲
نسبت آموزش، ارزیابی، آزمون	۸۰٪ - ۱۰٪ - ۱۰٪
بهینه‌ساز	Adam
نرخ یادگیری	۰,۰۹۸
تابع هزینه	MSE

نمودار کاهش های Loss داده‌های آموزشی، ارزیابی و آزمون به شرح زیر است.



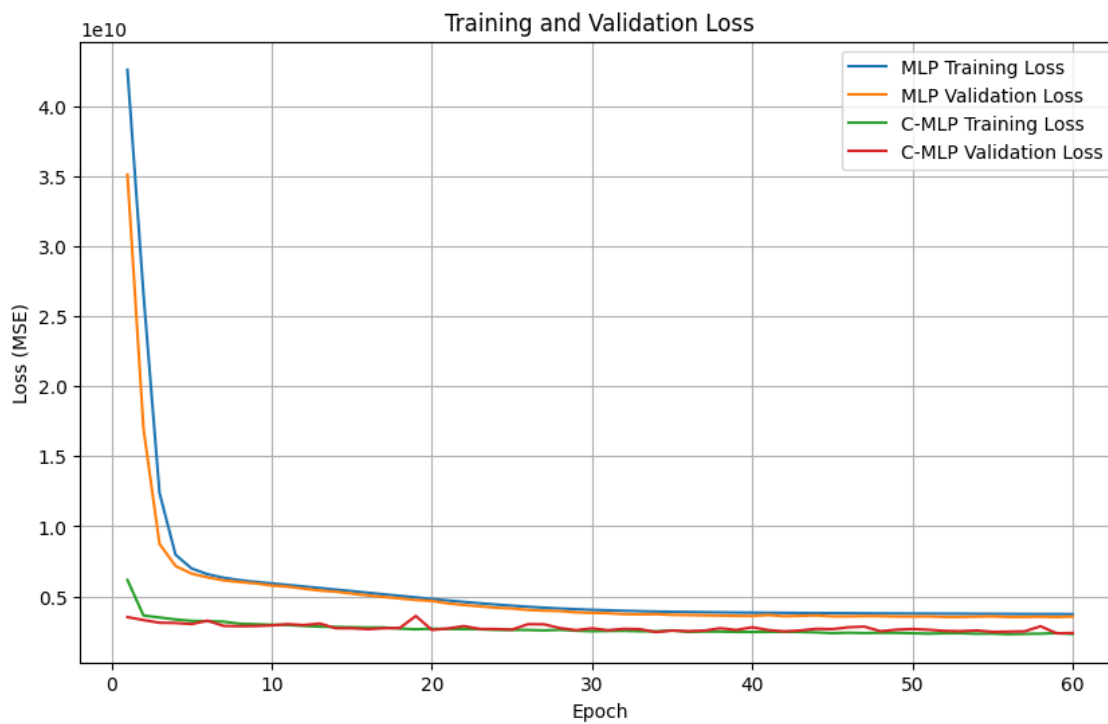
شکل ۱۷: نمودار کاهش خطا روی داده‌های آموزشی، ارزیابی و آزمون روی شبکه عصبی پیچیده

حال معیارهایی که توضیح دادیم را روی مدل شبکه عصبی پیچیدمتر محاسبه می‌کنیم.

جدول ۲۰: معیارهای سنجش روی شبکه عصبی پیچیدمتر

Dataset	MSE	RMSE	MAE	R ²
Train	۲۲۰۱۴۰۸۵۱۲.۰۰۰۰	۴۶۹۱۹.۱۷۱۹	۳۲۴۸۱.۴۸۲۴	۰.۷۵۸۷
Test	۲۲۸۹۳۵۲۷.۰۴.۰۰۰۰	۴۷۸۴۷.۱۷۹۷	۳۲۹۹۱.۲۷۷۳	۰.۷۵۰۰
Validation	۲۴۰۹۲۰۹۰۸۸.۰۰۰۰	۴۹۰۸۳.۶۹۵۳	۳۳۴۱۴.۴۷۶۶	۰.۷۳۰۳

۵-۳ مقایسه و تحلیل نمودارها



شکل ۱۸: مقایسه نمودار کاهش خطا بین دو مدل ساده و پیچیده

همانطور که می‌بینید مدل پیچیده‌تر خطای کمتری دارد نسبت به مدل ساده‌تر.

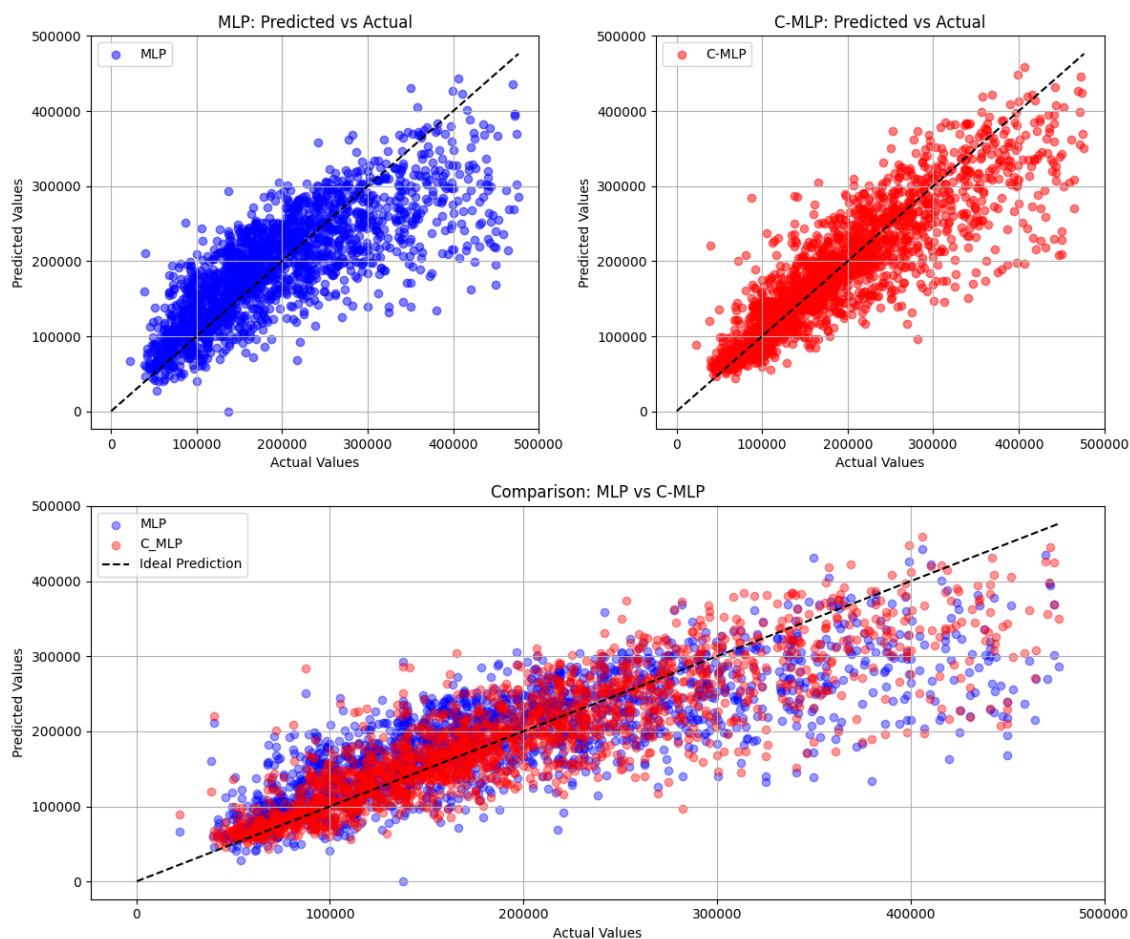
جدول ۲۱: مقایسه نتایج ارزیابی مدل‌ها بر روی داده آزمون

Metric	MLP	C-MLP
Test MSE	۳۴۹۳۶۴۱۴۷۲.۰۰۰۰	۲۲۸۹۳۵۲۷.۴.۰۰۰۰
Test RMSE	۵۹۱۰۷.۰۳۵۲	۴۷۸۴۷.۱۷۹۷
Test MAE	۴۴۰۶۵.۱۶۸۰	۳۲۹۹۱.۲۷۷۳
Test R ²	۰.۶۱۸۵	۰.۷۵۰۰

همانطور که از جدول بالا پیداست، تمامی معیارهای سنجش روی مدل پیچیده‌تر از مدل ساده‌تر بهتر است و

در نتیجه مدل پیچیده به وضوح از مدل ساده‌تر بهتر است.

۳ رگرشن با استفاده از mlp



شکل ۱۹: مقایسه مقدار واقعی با مقدار پیشبینی شده در دو مدل

همانطور که در نمودار بالا مشخص است، خط کج نشان‌دهنده این است که خروجی مدل عینا با برچسب داده مطابقت دارد. طبیعی است که هر چه خروجی مدل به برچسب نزدیکتر باشد (یا به عبارتی هر چه نقاط به این خط کج نزدیکتر باشند) مدل دقت بیشتری دارد. همانطور که از نمودار پائینی پیداست، نقاط قرمز به این خط بیشتر نزدیکتر هستند تا نقاط آبی؛ و این یعنی عملکرد مدل پیچیده‌تر بهتر از مدل ساده می‌باشد

۴ طراحی شبکه AutoEncoder

۱-۴ پیش‌پردازش

پرسش: تحلیل کنید چرا نرمال‌سازی داده‌ها اهمیت دارد.

نرمال‌سازی داده‌ها در یادگیری عمیق به دلایل متعددی اهمیت دارد:

- همگرایی پایدار و سریع‌تر: زمانی که ویژگی‌ها در مقیاس‌های مشابه قرار دارند، به‌روزرسانی‌های گرادیان نزولی به‌صورت یکنواخت روی تمام پارامترها انجام می‌شود و از نوسانات یا واگرایی جلوگیری می‌کند.
- جلوگیری از ناپایداری عددی: مقادیر بزرگ ورودی می‌توانند منجر به فعال‌سازی‌ها و گرادیان‌های بسیار بزرگ شوند که مشکل *exploding* یا *gradient vanishing* را ایجاد می‌کنند.
- بهبود تعمیم: داده‌های نرمال‌شده به شبکه کمک می‌کنند تا الگوهای عمومی را یاد بگیرد و کمتر بر مقیاس مقادیر ورودی تمرکز کند.
- سازگاری با توابع فعال‌سازی: توابعی مانند sigmoid، tanh یا ReLU بهترین عملکرد را زمانی دارند که ورودی‌ها حول صفر متمرکز بوده و اندازه کوچکی داشته باشند.

۲-۴ پیاده‌سازی

تحلیل و بررسی شبکه AutoEncoder

AutoEncoder یک شبکه عصبی غیرنظارتی (unsupervised) است که هدف آن یادگیری یک نمایش فشرده compressed representation از داده‌های ورودی است. شبکه شامل دو بخش اصلی می‌باشد:

- ۱ Encoder (رمزگذار): بخشی از شبکه که داده‌های ورودی X را به یک بردار با ابعاد کمتر نگاشت می‌کند.

$$Z = f_{\text{encoder}}(X)$$

- **۲ Decoder (رمزگشا):** بخشی که تلاش می‌کند ورودی اصلی را از بردار کد بازسازی کند:

$$\hat{X} = f_{\text{decoder}}(Z)$$

هدف شبکه کمینه کردن **خطای بازسازی** بین X و $\hat{X}$ است، معمولاً با استفاده از **Error Squared Mean**

(MSE):

$$\mathcal{L}(X, \hat{X}) = \frac{1}{N} \sum_{i=1}^N \|X_i - \hat{X}_i\|^2$$

شبکه AutoEncode مزایا و کاربردهایی دارد.

- **کاهش ابعاد:** شبکه‌های AutoEncode می‌توانند ویژگی‌های مهم را فشرده کنند و داده‌های با ابعاد بالا را به نمایش‌های کم‌بعدی تبدیل کنند.
- **کشف ویژگی‌های پنهان:** شبکه قادر است الگوهای پیچیده و روابط غیرخطی بین ویژگی‌ها را یاد بگیرد.
- **کاربردهای متعدد:** شامل بازسازی داده، حذف نویز Denoising AutoEncoder، تولید نمونه‌های جدید Variational AutoEncoder و پیش‌پردازش داده برای یادگیری عمیق.

یک معماری ساده AutoEncoder شامل موارد زیر است:

- لایه ورودی با ابعاد d .
- چند لایه پنهان Hidden Layers در Encoder برای کاهش ابعاد.
- لایه کد Bottleneck Layer با کمترین ابعاد که نمایش فشرده داده‌ها را نگه می‌دارد.
- چند لایه پنهان در Decoder برای بازسازی داده‌ها.
- لایه خروجی با ابعاد برابر با ورودی d .
- توابع فعال‌سازی غیرخطی مانند ReLU، tanh یا sigmoid برای یادگیری روابط غیرخطی.
- استفاده از گرادیان نزولی و Backpropagation برای آموزش شبکه.

معماری AutoEncoder برای این مسئله

معماری شبکه به صورت زیر است:

جدول ۲۲: معماری AutoEncoder

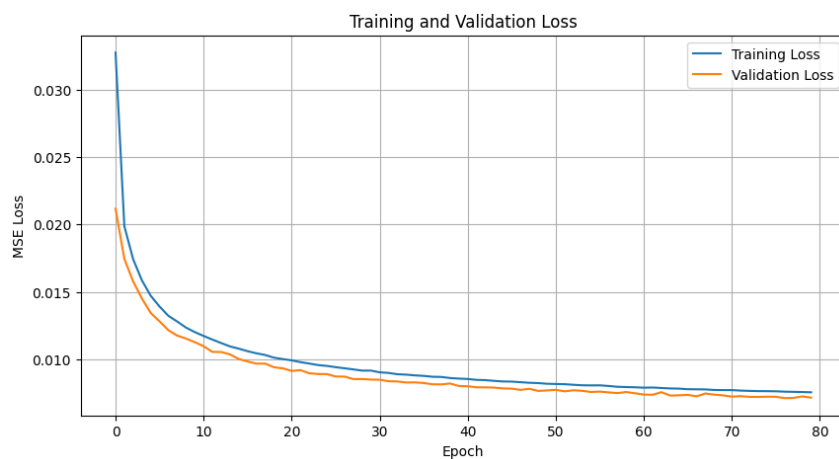
Layer	Type / Activation	Output Dimension
Input Layer	—	۷۸۴
Encoder Layer ۱	Linear + BatchNorm + ReLU + Dropout(0.2)	۵۱۲
Encoder Layer ۲	Linear + BatchNorm + ReLU	۲۵۶
Encoder Layer ۳	Linear + Tanh	۶۴ (encoding)
Decoder Layer ۱	Linear + ReLU	۲۵۶
Decoder Layer ۲	Linear + ReLU	۵۱۲
Decoder Layer ۳	Linear + Sigmoid	۷۸۴ (reconstructed output)

این شبکه را با هایپرپارامترهای زیر آموزش داده‌ایم:

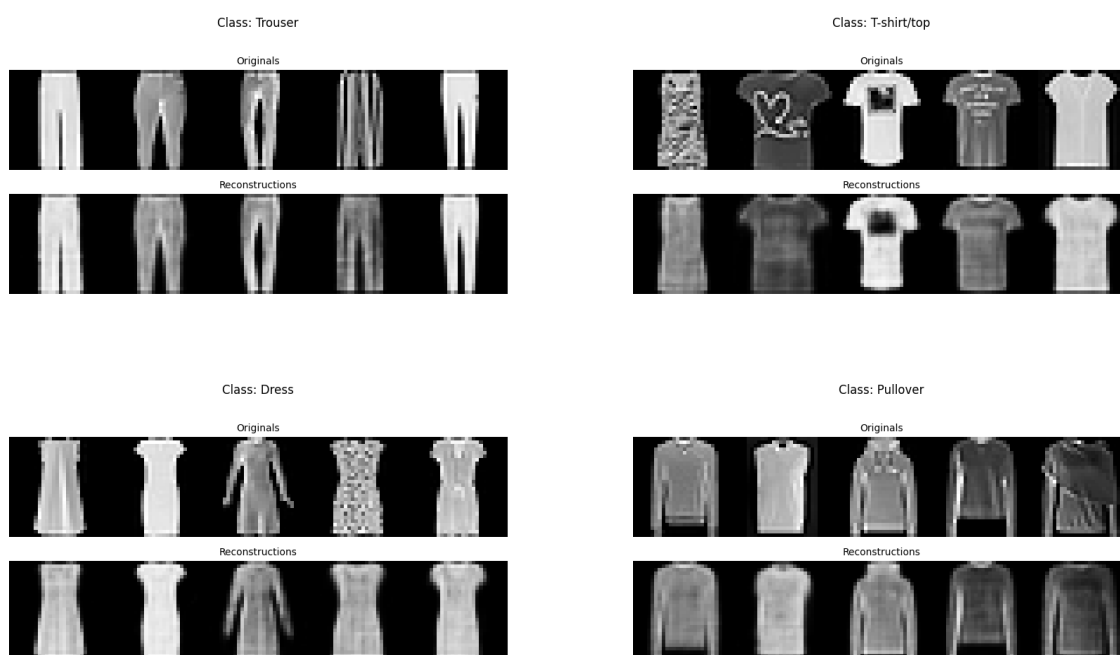
جدول ۲۳: هایپرپارامترهای مدل AutoEncoder

مقدار	هایپرپارامتر
۱۲۸	اندازه دسته (Batch Size)
۸۰٪ - ۲۰٪	نسبت آموزش، آزمون
AdamW	بهینه‌ساز
$5e-4$	نرخ یادگیری
MSE	تابع هزینه
80	epoch

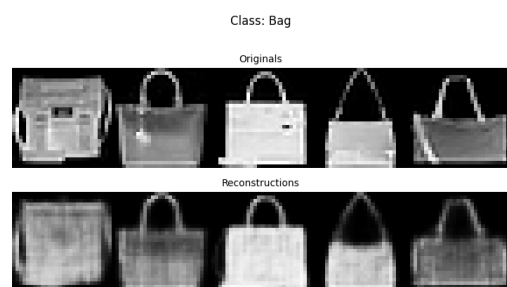
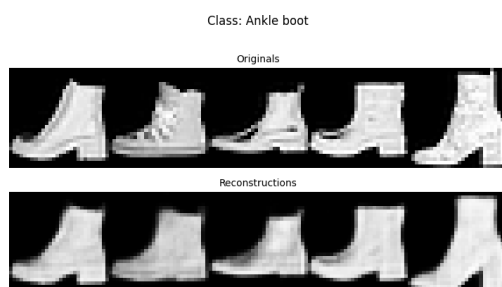
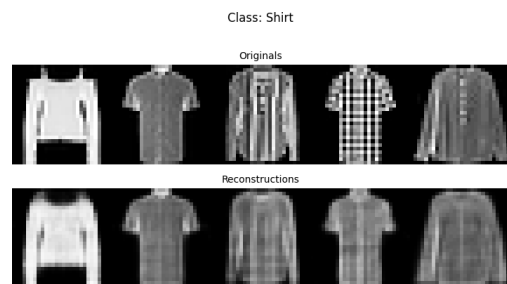
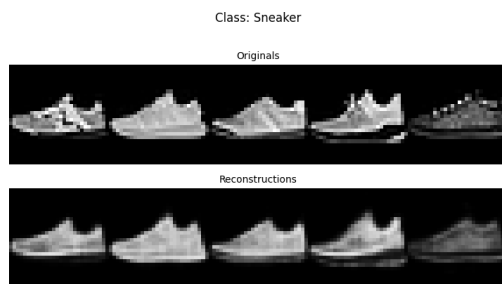
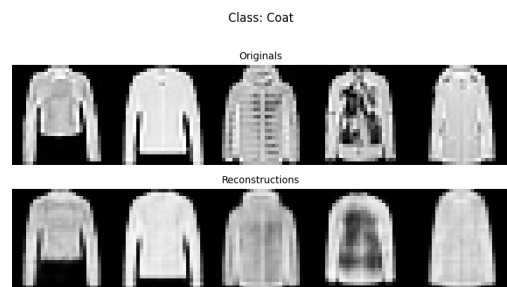
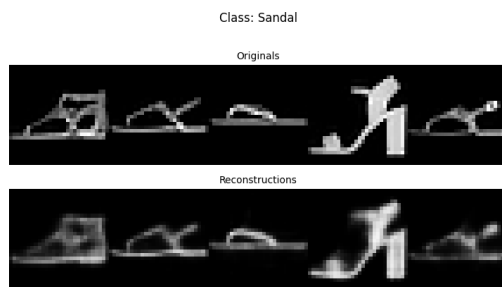
۳-۴ نتایج آموزش شبکه



شکل ۲۰: نمودار کاهش خطای شبکه AutoEncoder



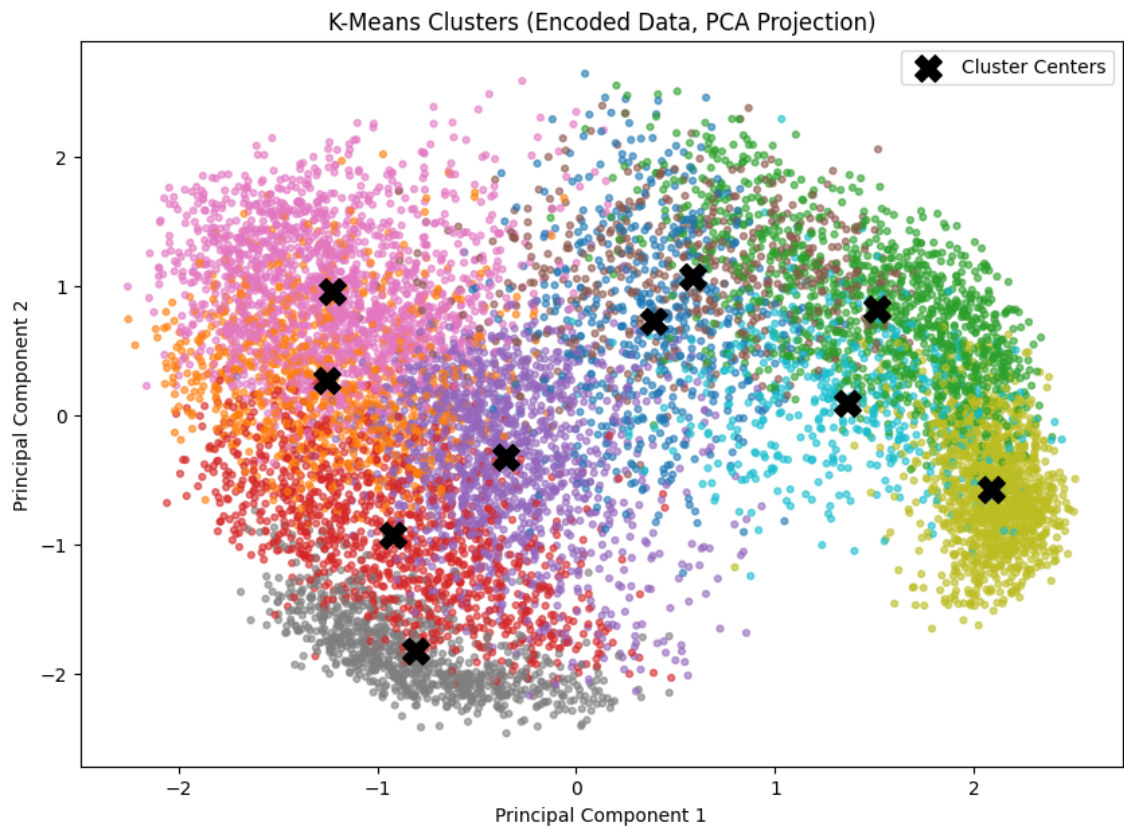
۴ طراحی شبکه AutoEncoder



همانطور که می‌بینید تصاویر بازسازی‌شده مقداری هموارتر شده و نویز آن گرفته شده است. ولی مقداری از جزئیات تصویر نیز کاسته شده است که نوعی کاهش اطلاعات می‌باشد.

۴-۴ ارزیابی شبکه

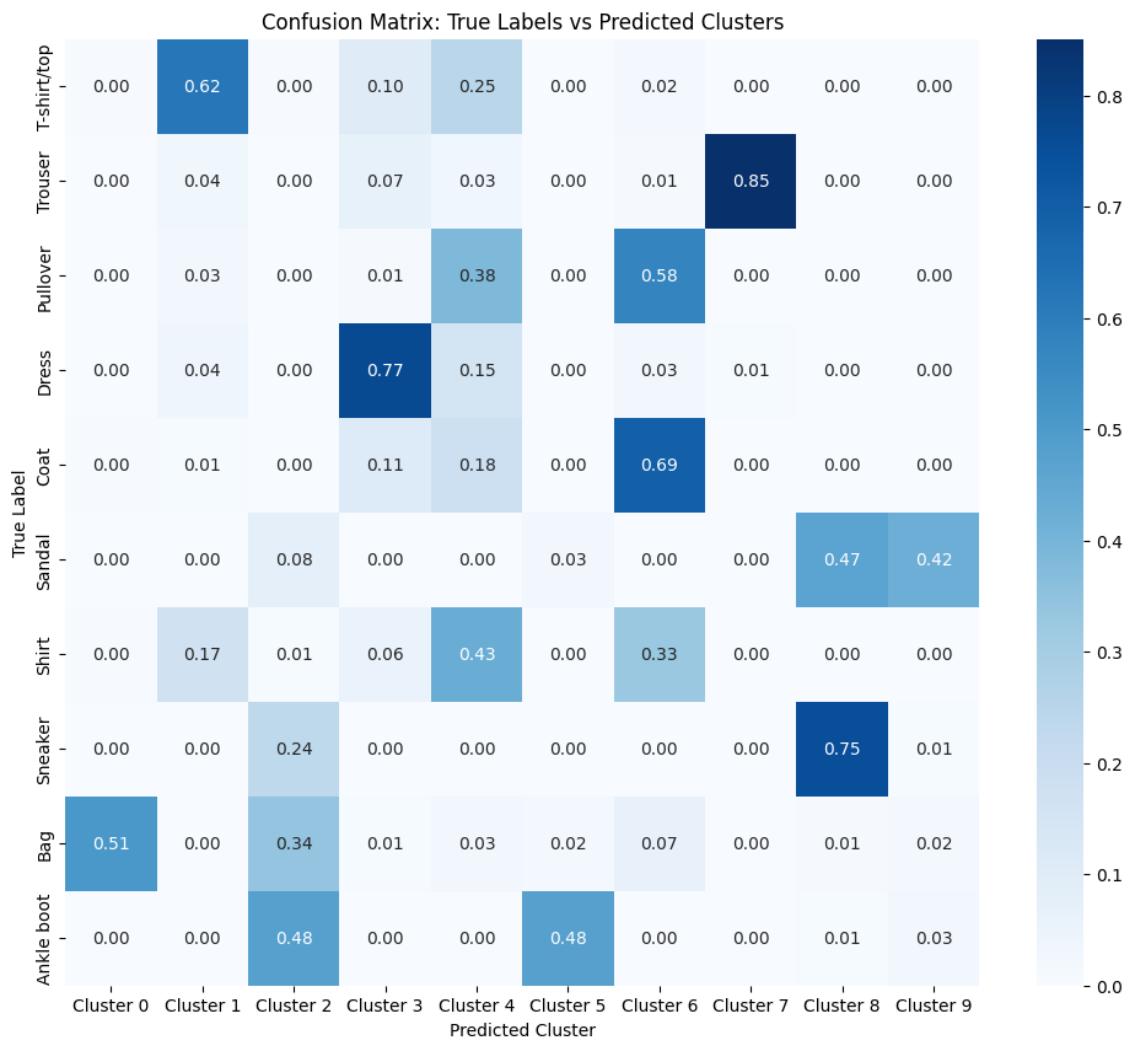
۱-۴-۴ خوشه‌بندی با الگوریتم K-Means



شکل ۲۱: K-means

۲-۴-۴ تحلیل خوشه‌ها

ماتریس درهم‌ریختگی بین خوشه‌ها و کلاس‌ها به صورت زیر است:



شکل ۲۲: ماتریس درهم‌ریختگی بین خوشه‌ها و برچسب‌ها

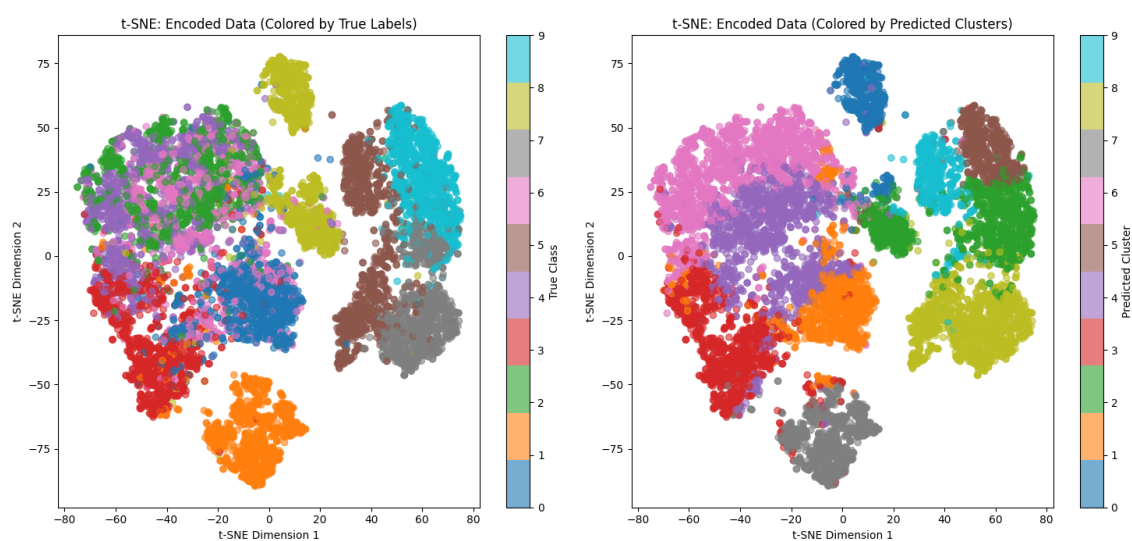
Analyzing clusters...

Cluster 0 mostly corresponds to: Bag (with 510 samples)
 Cluster 1 mostly corresponds to: T-shirt/top (with 619 samples)
 Cluster 2 mostly corresponds to: Ankle boot (with 482 samples)
 Cluster 3 mostly corresponds to: Dress (with 765 samples)
 Cluster 4 mostly corresponds to: Shirt (with 430 samples)
 Cluster 5 mostly corresponds to: Ankle boot (with 478 samples)
 Cluster 6 mostly corresponds to: Coat (with 694 samples)
 Cluster 7 mostly corresponds to: Trouser (with 851 samples)
 Cluster 8 mostly corresponds to: Sneaker (with 754 samples)
 Cluster 9 mostly corresponds to: Sandal (with 424 samples)

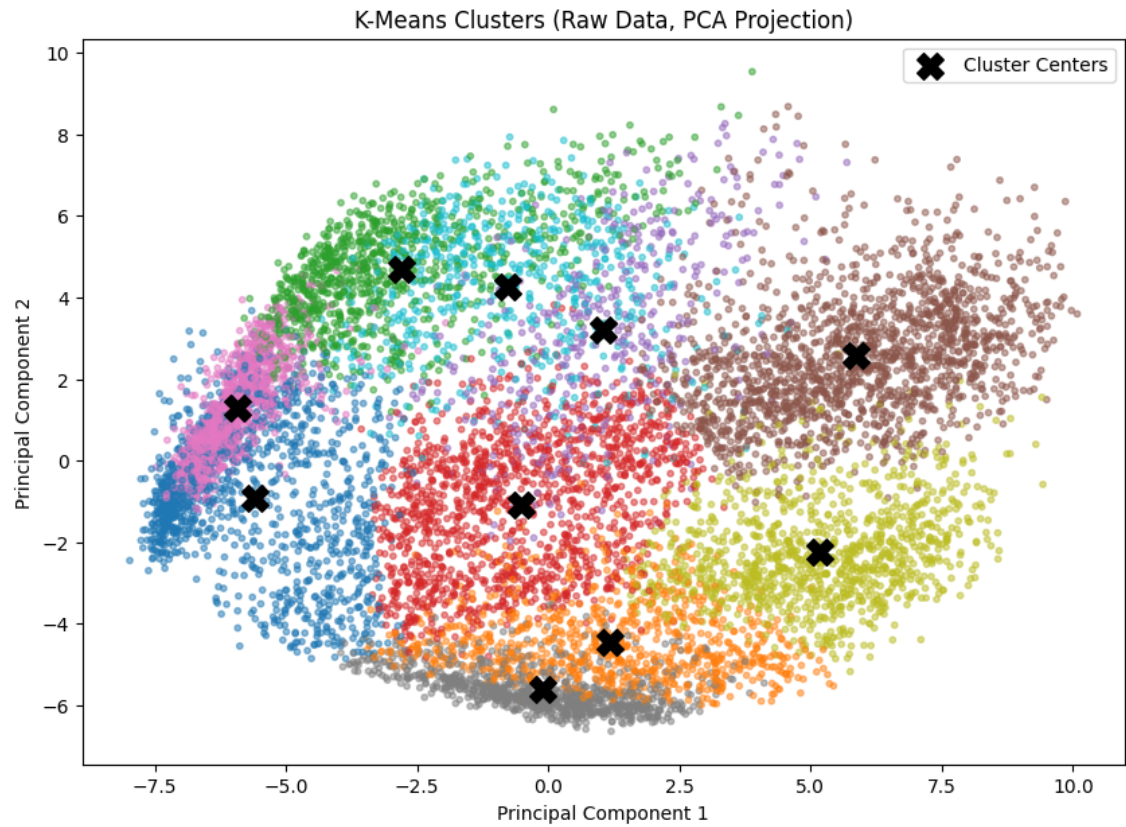
۴ طراحی شبکه AutoEncoder

با توجه به ماتریس درهم ریختگی در خوشه ۲، کلاس bag و boot ankle و sneaker اشتراک زیادی دارند در خوشه ۴، کلاس shirt و pullover اشتراک زیادی دارند در خوشه ۶، کلاس coat و pullover و shirt اشتراک زیادی دارند در خوشه ۸، کلاس sneaker و sandle اشتراک زیادی دارند.

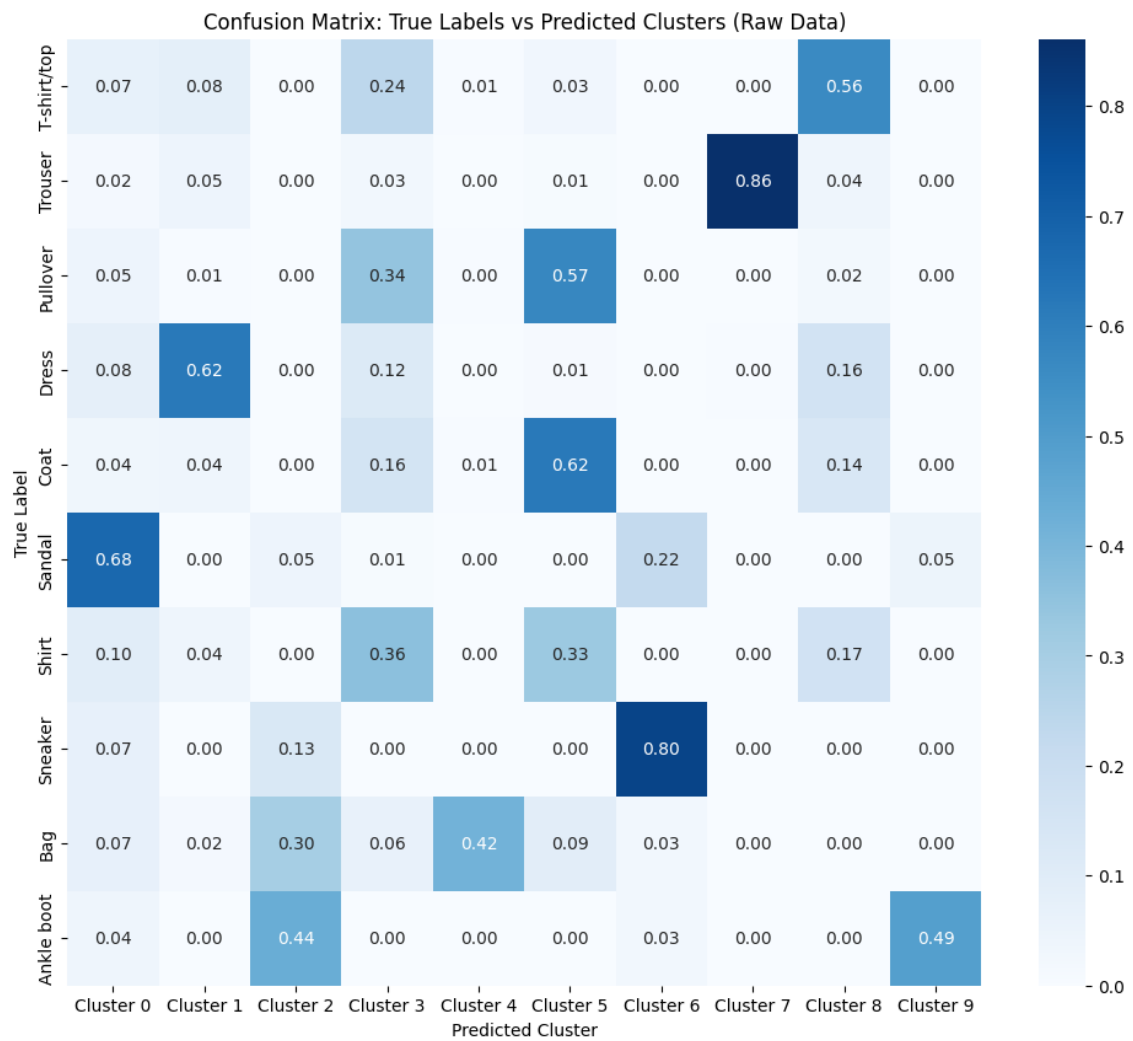
۴-۴-۳ کاهش ابعاد با t-SNE



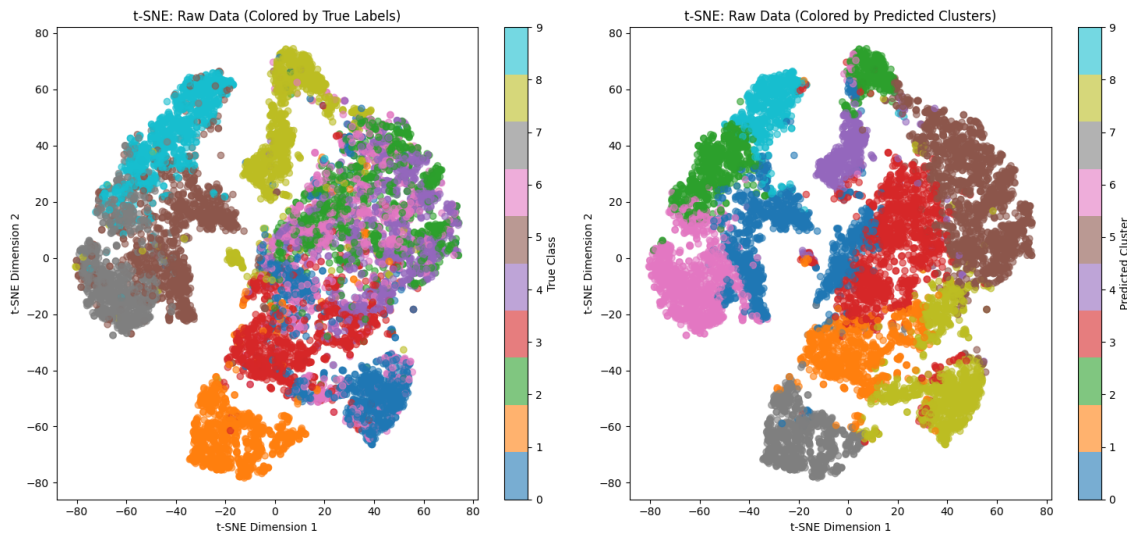
شکل ۲۳: t-SNE



شکل ۲۴: kmeans داده‌های خام



شکل ۲۵: ماتریس درهم‌ریختگی روی داده‌های خام



شکل ۲۶: t-SNE داده‌های خام

۴-۵ معیارهای ارزیابی کیفیت خوشه‌بندی

ARI میزان شباهت میان دو خوشه‌بندی را بر اساس تعداد جفت نمونه‌هایی که به‌صورت مشابه دسته‌بندی شده‌اند، اندازم‌گیری می‌کند. این شاخص اثر تصادف را نیز تعدیل می‌کند:

$$ARI = \frac{RI - \mathbb{E}[RI]}{\max(RI) - \mathbb{E}[RI]},$$

که در آن RI شاخص رند است. مقدار ARI بین ۱- تا ۱ قرار دارد که مقدار ۱ بیانگر تطابق کامل است.

AMI میزان اطلاعات مشترک بین دو خوشه‌بندی را با در نظر گرفتن تصادف می‌سنجد:

$$AMI = \frac{I(U; V) - \mathbb{E}[I(U; V)]}{\max\{H(U), H(V)\} - \mathbb{E}[I(U; V)]},$$

که در آن $I(U; V)$ اطلاعات متقابل و $H(U)$ و $H(V)$ آنترپی خوشه‌ها هستند. مقدار AMI نیز در بازه $[0, 1]$ قرار دارد و مقدار ۱ نشان‌دهنده‌ی انطباق کامل است.

مقدار AMI و ARI خوشه‌بندی‌ها به شرح زیر است:

۴ طراحی شبکه AutoEncoder

Clustering Quality Evaluation:

Encoded Data:

Adjusted Rand Index (ARI): 0.3992

Adjusted Mutual Information (AMI): 0.8590

Raw Data:

Adjusted Rand Index (ARI): 0.3917

Adjusted Mutual Information (AMI): 0.8445

Criteria Check:

Encoded Data ARI $\geq$:3.0 (0.3992)

Encoded Data AMI $\geq$:5.0 (0.8590)

Raw Data ARI $\geq$:3.0 (0.3917)

Raw Data AMI $\geq$:5.0 (0.8445)

Comparison:

ARI improvement: 0.0075

AMI improvement: 0.0145

در نتیجه استفاده از داده‌های Encoded شده بهتر از استفاده آن‌ها به صورت خام است.